



UNIVERSITATEA DE VEST DIN TIMIȘOARA
FACULTATEA DE INFORMATICĂ
PROGRAMUL DE STUDII DE MASTER: Bioinformatică

LUCRARE DE DIZERTATIE

COORDONATOR:
Prof. Dr. Zaharie Daniela

ABSOLVENT:
Gâță Răzvan-Andrei

TIMIȘOARA
2026

UNIVERSITATEA DE VEST DIN TIMIȘOARA
FACULTATEA DE INFORMATICĂ
PROGRAMUL DE STUDII DE MASTER: Bioinformatică

LUCRARE DE DIZERTAȚIE

Clasificarea Semnalelor EEG utilizând
Rețele Neuronale de tip Graf

COORDONATOR:
Prof. Dr. Zaharie Daniela

ABSOLVENT:
Gâță Răzvan-Andrei

TIMIȘOARA
2026

Abstract - Română

Electroencefalografia (EEG) reprezintă una dintre cele mai utilizate metode pentru monitorizarea activității electrice a creierului, datorită caracterului său non-invaziv, portabilității și rezoluției temporale ridicate. Cu toate acestea, analiza semnalelor EEG rămâne o provocare majoră, deoarece acestea sunt serii de timp multicanal complexe, afectate de zgomot, artefacte și variabilitate ridicată între subiecți. În plus, structura lor non-euclidiană limitează aplicabilitatea metodelor convenționale de învățare profundă, precum rețelele neuronale convoluționale.

Prezenta lucrare investighează utilizarea rețelelor neuronale de tip graf (Graph Neural Networks – GNN) pentru analiza și clasificarea datelor EEG, cu accent pe captarea relațiilor complexe dintre diferitele regiuni cerebrale. În cadrul abordării propuse, semnalele EEG sunt reprezentate sub forma unor grafuri, în care electrozii sunt modelați ca noduri, iar relațiile dintre aceștia sunt descrise prin muchii. Această reprezentare permite păstrarea topologiei datelor și integrarea informațiilor spațiale și funcționale într-un mod mai adecvat decât metodele bazate pe structuri regulate.

Lucrarea include o analiză teoretică a particularităților datelor EEG, a etapelor de pre-procesare necesare și a principalelor arhitecturi GNN relevante pentru acest domeniu, în special Graph Convolutional Networks (GCN) și Graph Attention Networks (GAT). De asemenea, este propus și implementat un pipeline complet pentru procesarea datelor EEG, construirea reprezentării de tip graf, antrenarea și evaluarea modelelor, utilizând limbajul Python și biblioteca PyTorch Geometric. În plus, a fost dezvoltată o interfață grafică dedicată vizualizării și interpretării rezultatelor obținute.

Motivația principală a lucrării este dată de potențialul utilizării GNN în aplicații biomedicale, în special în contextul identificării unor tipare asociate afecțiunilor neurologice și neurodegenerative, precum boala Alzheimer. Prin această lucrare se urmărește evidențierea avantajelor oferite de modelele bazate pe graf în analiza semnalelor EEG și susținerea utilizării acestora ca alternativă promițătoare la abordările tradiționale.

Abstract - English

Electroencephalography (EEG) is one of the most widely used techniques for monitoring brain activity, due to its non-invasive nature, portability, and high temporal resolution. However, the analysis of EEG signals remains a challenging task, as these signals are complex multichannel time series affected by noise, artifacts, and high inter-subject variability. Moreover, their non-Euclidean structure limits the applicability of conventional deep learning approaches, such as convolutional neural networks.

This thesis investigates the use of Graph Neural Networks (GNN) for the analysis and classification of EEG data, with a focus on capturing the complex relationships between different brain regions. In the proposed approach, EEG signals are represented as graphs, where electrodes are modeled as nodes and the relationships between them are represented as edges. This representation enables the preservation of the intrinsic structure of the data and allows for a more effective integration of spatial and functional information.

The work includes a theoretical analysis of EEG data characteristics, preprocessing techniques, and the main GNN architectures relevant to this domain, particularly Graph Convolutional Networks (GCN) and Graph Attention Networks (GAT). Furthermore, a complete pipeline is proposed and implemented for EEG data processing, graph construction, model training, and evaluation, using Python and the PyTorch Geometric library. In addition, a graphical user interface is developed to facilitate the visualization and interpretation of the obtained results.

The main motivation of this study lies in the potential of GNN-based approaches in biomedical applications, especially for identifying patterns associated with neurological and neurodegenerative disorders, such as Alzheimer’s disease. This thesis aims to highlight the advantages of graph-based models in EEG signal analysis and to support their use as a promising alternative to traditional methods.

Notății și abrevieri

În cuprinsul prezentei lucrări sunt utilizate în mod consistent următoarele notații matematice și abrevieri:

Notății matematice generale

- i – unitatea imaginară, cu proprietatea $i^2 = -1$
- $\mathbb{R}^{m \times n}$ – spațiul matricelor reale de dimensiune $m \times n$
- $|\cdot|$ – modulul unui număr complex sau valoarea absolută a unui număr real
- $\arg(\cdot)$ – argumentul (faza) unui număr complex
- $\lfloor \cdot \rfloor$ – partea întreagă inferioară (floor)
- $\| \cdot \|$ – operația de concatenare a vectorilor (în context GNN)

Notății specifice semnalelor EEG și conectivității funcționale

- $x(t)$ – semnalul EEG real, în funcție de timp t
- $z(t)$ – semnalul analitic complex asociat lui $x(t)$
- $\mathcal{H}[\cdot]$ – transformata Hilbert a unui semnal real
- $\phi(t)$ – faza instantanee a semnalului $x(t)$
- $\phi_i(t), \phi_j(t)$ – fazele instantanee ale canalelor EEG i , respectiv j
- T – numărul de eșantioane temporale ale unei epoci EEG
- $PLV_{ij} \in [0, 1]$ – Phase Locking Value dintre canalele i și j
- f_l, f_h – frecvența inferioară, respectiv superioară a unui filtru trece-bandă (Hz)
- f_s – rata de eșantionare a semnalului EEG (Hz)
- Δt – durata unei epoci EEG (secunde)
- $\mathbf{X} \in \mathbb{R}^{N_e \times N_c \times N_t}$ – tensorul epocilor EEG, cu N_e epoci, N_c canale și N_t eșantioane temporale

Notății specifice rețelelor neuronale de tip graf

- $G = (V, E)$ – graful EEG, unde V este mulțimea nodurilor (electrozi), iar E este mulțimea muchiilor (relații funcționale)
- $N = |V|$ – numărul de noduri ale grafului ($N = 19$ în implementarea prezentă)
- $A \in \mathbb{R}^{N \times N}$ – matricea de adiacență a grafului
- $\tilde{A} = A + I_N$ – matricea de adiacență cu auto-conexiuni adăugate, unde I_N este matricea identitate de dimensiune N

- $\tilde{D}$ – matricea diagonală de grade asociată lui $\tilde{A}$, cu $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$
- $\mathbf{h}_v^{(l)} \in \mathbb{R}^{d_l}$ – vectorul de caracteristici al nodului v la stratul l , de dimensiune d_l
- $\mathcal{N}(v)$ – mulțimea vecinilor nodului v în graf
- $W^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$ – matricea de ponderi antrenabilă a stratului l
- $\sigma(\cdot)$ – funcție de activare neliniară (e.g., ReLU sau ELU)
- α_{ij} – coeficientul de atenție normalizat al muchiei (i, j) în GAT
- e_{ij} – scorul de atenție nenormalizat al muchiei (i, j) în GAT
- $\mathbf{a} \in \mathbb{R}^{2d'}$ – vectorul de atenție antrenabil în GAT
- K – numărul de capuri de atenție (attention heads) în mecanismul multi-head al GAT

Abrevieri

- **EEG** – Electroencefalografie / Electroencephalography
- **GNN** – Rețea Neuronală de tip Graf / Graph Neural Network
- **GCN** – Rețea Convoluțională pe Graf / Graph Convolutional Network
- **GAT** – Rețea de Atenție pe Graf / Graph Attention Network
- **PLV** – Phase Locking Value (Valoarea de Blocare a Fazei)
- **ICA** – Analiza Componentelor Independente / Independent Component Analysis
- **FIR** – Finite Impulse Response (Răspuns Finit la Impuls)
- **PSD** – Power Spectral Density (Densitatea Spectrală de Putere)
- **PyG** – PyTorch Geometric
- **MNE** – biblioteca MNE-Python pentru procesarea semnalelor neurofiziologice
- **BIDS** – Brain Imaging Data Structure
- **BCI** – Brain-Computer Interface (Interfață Creier-Calculator)
- **CNN** – Rețea Neuronală Convoluțională / Convolutional Neural Network
- **ReLU** – Rectified Linear Unit (funcție de activare)
- **ELU** – Exponential Linear Unit (funcție de activare)

Cuprins

Notatii și abrevieri	5
1 Introducere	10
1.1 Motivație și context general	10
1.2 Obiective	11
1.3 Contribuții	11
1.4 Abordarea propusă	12
1.5 Structura lucrării	12
2 Procesarea datelor de tip EEG	14
2.1 Particularități ale datelor și descrierea problemei	14
2.1.1 Benzile de frecvență EEG	15
2.2 Tehnici de pre-procesare	16
2.3 Conectivitate funcțională și Phase Locking Value	17
2.3.1 Transformata Hilbert și faza instantanee	18
2.3.2 Calculul PLV	18
2.4 Rețelele neuronale de tip graf	19
2.4.1 Particularități ale GNN și mecanismul de message passing	19
2.4.2 Graph Convolutional Networks (GCN)	21
2.4.3 Graph Attention Networks (GAT)	21
2.4.4 Relevanța arhitecturilor GCN și GAT pentru analiza EEG	23
2.5 Procesarea semnalelor EEG utilizând rețele neuronale de tip graf	24
3 Implementare	26
3.1 Setul de date utilizat	26
3.1.1 Formatul și structura datelor	26
3.1.2 Configurația claselor de clasificare	27
3.1.3 Selecția canalelor EEG	27
3.2 Pre-procesarea semnalelor EEG	28
3.2.1 Încărcarea fișierelor EEG	28
3.2.2 Filtrarea trece-bandă	28
3.2.3 Curățarea prin ICA	29
3.2.4 Segmentarea în epoci	29
3.2.5 Conversia la format NumPy	30
3.3 Construirea reprezentărilor de tip graf	31
3.3.1 Extragerea trăsăturilor nodurilor	31
3.3.2 Calculul matricei de conectivitate funcțională	32
3.3.3 Aplicarea pragului de adiacență	32

3.3.4	Conversia la formatul PyTorch Geometric	33
3.3.5	Construirea obiectului Data	33
3.4	Construirea setului de date	34
3.4.1	Citirea metadatelor și maparea etichetelor	34
3.4.2	Procesarea per subiect	34
3.4.3	Raportarea statisticilor și salvarea pe disc	35
3.4.4	Separarea subiect-nivel pentru antrenare și validare	35
3.5	Arhitecturile modelelor implementate	36
3.5.1	Elemente comune ale arhitecturilor	36
3.5.2	Modelul bazat pe Graph Convolutional Networks	37
3.5.3	Modelul bazat pe Graph Attention Networks	38
3.5.4	Selecția dinamică a modelului	39
3.5.5	Comparație între arhitecturile implementate	40
3.6	Procesul de antrenare	40
3.6.1	Încărcarea și validarea setului de date	41
3.6.2	Separarea datelor în seturi de antrenare și validare	41
3.6.3	Inițializarea modelului și a componentelor de optimizare	42
3.6.4	Bucla de antrenare	43
3.6.5	Evaluarea pe setul de validare	43
3.6.6	Salvarea checkpoint-urilor și selecția celui mai bun model	44
3.6.7	Configurația experimentală salvată	45
3.7	Mecanismul de inferență	45
3.7.1	Încărcarea checkpoint-ului și reconstrucția modelului	46
3.7.2	Pre-procesarea unei înregistrări noi	46
3.7.3	Predicția la nivel de fereastră temporală	46
3.7.4	Agregarea predicțiilor la nivel de subiect	47
3.7.5	Maparea etichetelor numerice la denumiri clinice	48
3.7.6	Rolul inferenței în aplicația finală	48
4	Interfața grafică	49
4.1	Prezentare generală	50
4.2	Vizualizarea semnalelor EEG	51
4.3	Configurarea și antrenarea modelelor	53
4.4	Realizarea predicțiilor	54
4.5	Observații privind utilitatea interfeței	57
5	Rezultate experimentale	58
5.1	Configurația experimentală	58
5.2	Rezultate la nivel de fereastră temporală	59
5.2.1	Clasificare binară (Healthy Control vs. Alzheimer's Disease)	59
5.2.2	Clasificare pe trei clase (HC vs. AD vs. FTD)	59
5.3	Rezultate la nivel de subiect (inferență pe holdout)	59
5.4	Analiză comparativă și discuții	60
6	Concluzii și direcții viitoare de dezvoltare	62
6.1	Concluzii	62
6.2	Limitări identificate	63
6.3	Direcții viitoare de dezvoltare	64

Capitolul 1

Introducere

Prezentul capitol introduce contextul general al lucrării, descriind motivația utilizării rețelelor neuronale de tip graf pentru analiza semnalelor EEG, obiectivele urmărite, contribuțiile principale și abordarea generală adoptată.

1.1 Motivație și context general

În domeniul neuroștiințelor, înțelegerea activității neuronale și a modului în care aceasta corelează cu comportamentul și funcțiile cognitive este esențială, atât pentru diagnosticarea și tratarea afecțiunilor neurologice, precum Alzheimer sau epilepsie, cât și pentru dezvoltarea aplicațiilor de tip BCI (Brain-Computer Interfaces). În acest context, având acces la un volum tot mai mare de date medicale, metodele de învățare automată și inteligență artificială devin o soluție indispensabilă pentru extragerea informațiilor relevante și realizarea unor predicții precise [1].

Standardul de aur pentru monitorizarea activității electrice a creierului este electroencefalografia (EEG). Aceasta s-a impus ca metodă preferată deoarece este non-invazivă, portabilă și are un cost redus comparativ cu alte tehnici. În plus, oferă o rezoluție temporală excelentă, permițând astfel monitorizarea schimbărilor la nivel de milisekunde. Spre deosebire de alte metode de imagistică cerebrală precum fMRI sau PET care oferă o rezoluție temporală mult mai slabă, EEG-ul poate surprinde viteza reală de procesare a creierului [2]. Acest lucru face ca datele EEG să fie extrem de valoroase pentru înțelegerea dinamicii creierului și pentru dezvoltarea unor aplicații precum BCI sau pentru diagnosticarea precoce a afecțiunilor neurologice.

Cu toate acestea, semnalele EEG sunt serii de timp multicanal extrem de complexe, adesea contaminate de zgomot și artefacte fiziologice (precum mișcări oculare, activitate musculară sau clipit), dar și de interferențe electromagnetice. Analiza manuală a rezultatelor este o sarcină laborioasă pentru specialiști deoarece consumă mult timp, este subiectivă și este predispusă la erori umane, mai ales în cazul monitorizărilor de lungă durată [3]. În consecință, apare o nevoie acută de automatizare a procesului de interpretare a datelor EEG.

Metodele clasice de învățare automată, precum Random Forest sau Support Vector Machines, au fost utilizate în trecut pentru analiza datelor EEG. Totuși, acestea depind în mare măsură de extragerea manuală a caracteristicilor relevante (feature engineering), ceea ce limitează capacitatea lor de a surprinde dinamica complexă a activității cerebrale. Ulterior, apariția tehnicilor de învățare profundă (deep learning)

a permis extragerea automată a caracteristicilor din date brute și modelarea relațiilor complexe din semnale [4].

Majoritatea modelelor de deep learning utilizate, în special rețelele neuronale convoluționale (CNN), sunt concepute pentru date cu structură euclidiană, precum imaginile, unde informația este organizată pe o grilă regulată. În schimb, datele EEG au o natură non-euclidiană, electrozii fiind distribuiți neregulat pe scalp, iar relațiile dintre aceștia fiind complexe și dependente de funcționalitatea cerebrală, nu doar de proximitatea fizică. Această discrepanță limitează capacitatea modelelor tradiționale de a captura în mod eficient interdependențele dintre diferite regiuni ale creierului.

În acest context, rețelele neuronale graf (Graph Neural Networks - GNN) reprezintă o direcție promițătoare pentru analiza datelor EEG. Prin modelarea electrozilor ca noduri într-un graf și a relațiilor dintre aceștia ca muchii, GNN-urile permit captarea structurii intrinseci a datelor și a interacțiunilor complexe dintre regiuni cerebrale. Astfel, utilizarea acestor modele oferă un cadru mai adecvat pentru analiza semnalelor EEG și poate conduce la îmbunătățirea performanței în sarcini precum clasificarea sau detecția afecțiunilor neurologice.

1.2 Obiective

Obiectivul principal al prezentei lucrări constă în dezvoltarea și evaluarea unui model bazat pe rețele neuronale de tip graf (GNN) pentru analiza și clasificarea datelor EEG, în vederea evidențierii relațiilor complexe dintre diferitele regiuni cerebrale. De asemenea, se urmărește dezvoltarea unei interfețe grafice care să permită vizualizarea și interpretarea rezultatelor obținute în urma antrenării și testării modelului.

Pentru atingerea acestui obiectiv general, au fost definite următoarele obiective specifice:

- Preprocesarea semnalelor EEG și reducerea artefactelor;
- Transformarea datelor obținute într-o reprezentare de tip graf, prin modelarea electrozilor ca noduri și a relațiilor dintre aceștia ca muchii;
- Implementarea unui model de rețea neuronală graf utilizând biblioteca PyTorch Geometric;
- Antrenarea și validarea modelului pe seturi de date specifice;
- Evaluarea performanței modelului utilizând metrici relevante precum acuratețea, precizia și scorul F1;
- Dezvoltarea unei interfețe pentru vizualizarea și interpretarea rezultatelor obținute.

1.3 Contribuții

1. Implementarea și evaluarea comparativă a două arhitecturi de rețele neuronale graf – Graph Convolutional Network (GCN) și Graph Attention Network (GAT) – pentru clasificarea semnalelor EEG asociate afecțiunilor neurodegenerative;

2. Dezvoltarea unui sistem integrat care permite atât antrenarea și evaluarea modelelor, cât și realizarea de predicții pe înregistrări noi, într-o manieră accesibilă prin intermediul unei interfețe grafice interactive bazate pe Streamlit.

1.4 Abordarea propusă

Pentru a depăși limitările impuse de abordările convenționale, în special ale rețelelor neuronale convoluționale, în analiza datelor EEG, în cadrul acestei lucrări se propune utilizarea rețelelor neuronale graf (Graph Neural Networks - GNN). Această alegere este motivată de natura non-euclidiană a datelor EEG, care nu pot fi reprezentate eficient pe o structură de tip grilă.

Semnalele obținute în urma unei electroencefalografii pot fi modelate în mod natural sub forma unui graf. În această reprezentare, electrozii sunt considerați noduri, iar relațiile dintre aceștia, cum ar fi distanța fizică sau corelațiile funcționale, sunt modelate prin muchii. Această abordare permite păstrarea topologiei reale a creierului și captarea interdependențelor complexe dintre diferitele regiuni cerebrale, indiferent de proximitatea lor fizică.

Procesul propus în prezenta lucrare urmează un pipeline bine definit. În prima etapă, semnalele EEG sunt supuse unui proces de preprocesare, care include eliminarea zgomotului și a artefactelor, precum și alte etape specifice de pregătire a datelor. Ulterior, datele sunt transformate într-o reprezentare de tip graf, prin definirea nodurilor și a muchiilor, precum și a caracteristicilor asociate fiecărui nod. În etapa următoare, este implementat un model de rețea neuronală graf, capabil să proceseze aceste date și să învețe relațiile dintre noduri. Modelul este apoi antrenat și validat pe seturi de date specifice, iar performanța acestuia este evaluată utilizând metrici relevante pentru sarcina de clasificare.

Pentru implementarea modelului și a întregului pipeline a fost utilizat limbajul de programare Python, împreună cu biblioteca PyTorch Geometric (PyG), o soluție de ultimă generație pentru dezvoltarea aplicațiilor bazate pe rețele neuronale graf [5]. Aceasta oferă suport eficient pentru reprezentarea și procesarea grafurilor, precum și pentru antrenarea modelelor GNN.

În plus, în cadrul lucrării a fost dezvoltată o interfață grafică ce permite vizualizarea rezultatelor obținute în urma antrenării și testării modelului. Aceasta facilitează interpretarea predicțiilor și oferă utilizatorului posibilitatea de a configura parametrii analizei și de a studia comportamentul modelului într-un mod intuitiv și interactiv.

Prin această abordare, se urmărește valorificarea avantajelor oferite de rețelele neuronale graf în modelarea relațiilor complexe din datele EEG, oferind o alternativă mai adecvată față de metodele tradiționale bazate pe structuri euclidiene.

1.5 Structura lucrării

Lucrarea este structurată în șase capitole, fiecare abordând o etapă distinctă a procesului de analiză a datelor EEG utilizând rețele neuronale graf.

Capitolul 1 – Introducere prezintă contextul general al lucrării, motivația, obiectivele propuse, contribuțiile principale și abordarea adoptată.

Capitolul 2 – Procesarea datelor de tip EEG detaliază fundamentele teoretice necesare înțelegerii lucrării, incluzând particularitățile datelor EEG, tehnicile de pre-

procesare, conceptele de bază ale rețelelor neuronale graf (GCN și GAT), măsurile de conectivitate funcțională și aplicațiile GNN în analiza semnalelor EEG.

Capitolul 3 – Implementare descrie în detaliu pipeline-ul complet implementat, de la setul de date utilizat și etapele concrete de pre-procesare, la construirea reprezentărilor de tip graf, arhitecturile modelelor implementate, procesul de antrenare și mecanismul de inferență.

Capitolul 4 – Interfața grafică prezintă aplicația interactivă dezvoltată cu Streamlit, care permite vizualizarea semnalelor EEG, explorarea grafurilor de conectivitate funcțională, configurarea și lansarea antrenării modelelor, precum și realizarea de predicții pe înregistrări noi.

Capitolul 5 – Rezultate experimentale conține evaluarea performanței modelelor implementate, incluzând comparația dintre arhitecturile GCN și GAT, analiza impactului hiperparametrilor și discuții asupra rezultatelor obținute.

Capitolul 6 – Concluzii și direcții viitoare sintetizează contribuțiile lucrării, evidențiază limitările identificate și propune direcții de dezvoltare viitoare.

Capitolul 2

Procesarea datelor de tip EEG

Prezentul capitol prezintă fundamentele teoretice necesare înțelegerii lucrării: particularitățile datelor EEG și structura problemei de clasificare, tehnicile de pre-procesare aplicate, mecanismele de conectivitate funcțională, principalele arhitecturi GNN relevante și o trecere în revistă a utilizării GNN în analiza semnalelor EEG.

2.1 Particularități ale datelor și descrierea problemei

Semnalele înregistrate prin electroencefalografie (EEG) măsoară activitatea electrică a creierului prin intermediul unor electrozi plasați pe scalp. Aceste semnale sunt serii de timp multicanal, în care fiecare canal corespunde unui electrod și reflectă activitatea unei regiuni specifice a creierului [2].

Aceste date prezintă o serie de particularități care le diferențiază semnificativ de alte tipuri de date utilizate în mod obișnuit în învățarea automată. În primul rând, semnalele EEG sunt caracterizate printr-o rezoluție temporală ridicată, ceea ce permite captarea dinamicii rapide a activității cerebrale. În același timp, acestea sunt multicanal deoarece implică înregistrarea simultană a semnalelor din mai multe regiuni ale creierului, ceea ce conduce la existența unor dependențe complexe între canale.

Un alt aspect important îl reprezintă variabilitatea ridicată a semnalelor EEG, atât între diferiți subiecți, cât și în cadrul aceluiași subiect, în funcție de starea cognitivă sau condițiile experimentale. În plus, aceste semnale sunt puternic afectate de zgomot și artefacte, precum mișcările oculare, activitatea musculară sau interferențe electromagnetice, ceea ce îngreunează procesul de analiză și interpretare. De asemenea, semnalele EEG sunt adesea non-staționare, ceea ce înseamnă că proprietățile lor statistice se pot modifica în timp [3].

Din punct de vedere structural, datele EEG nu respectă o organizare de tip euclidian, specifică imaginilor sau altor date reprezentate pe grile regulate. În cazul imaginilor, pixelii sunt aranjați într-o structură fixă, iar relațiile dintre aceștia sunt locale și bine definite. În schimb, în cazul semnalelor EEG, electrozii sunt distribuiți neregulat pe suprafața scalpului, iar relațiile dintre aceștia nu depind exclusiv de proximitatea fizică, ci și de conexiunile funcționale dintre diferitele regiuni ale creierului [2, 4].

În Figura 2.1 este ilustrată diferența dintre datele Euclidiene și non-Euclidiene. În timp ce datele de tip imagine pot fi reprezentate sub forma unei grile regulate (A), semnalele EEG sunt mai bine descrise prin structuri de tip graf (B), în care nodurile reprezintă electrozii, iar muchiile reflectă relațiile dintre aceștia.

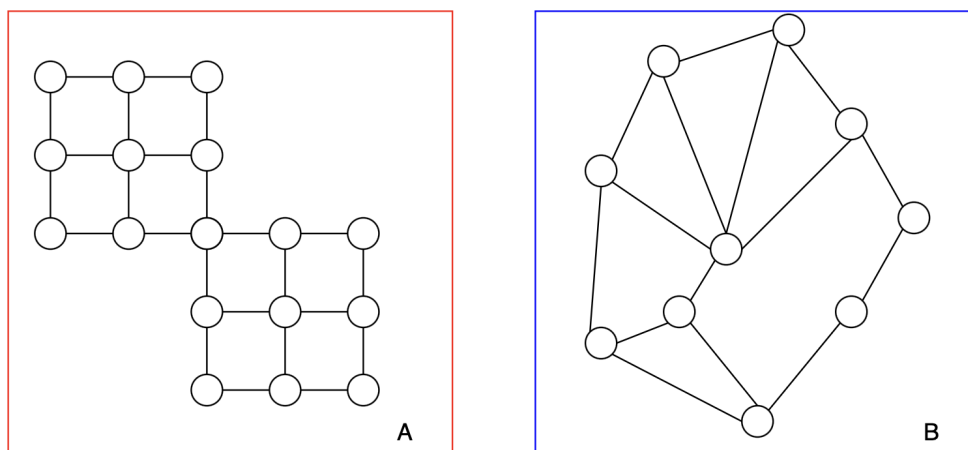


Figura 2.1: Reprezentarea structurii datelor: (A) Date Euclidiene (imagini) vs. (B) Date Non-Euclidiene (semnale EEG).

Această natură non-Euclidiană a datelor EEG limitează aplicabilitatea metodelor tradiționale de învățare automată și chiar a anumitor modele de deep learning, precum rețelele neuronale convoluționale, care sunt concepute pentru date cu structură regulată. În consecință, apare necesitatea utilizării unor modele capabile să gestioneze structuri de date complexe și neregulate.

În cadrul acestei lucrări, problema abordată constă în clasificarea semnalelor EEG, utilizând o reprezentare de tip graf și modele bazate pe rețele neuronale graf. Această abordare permite captarea relațiilor complexe dintre canale și oferă un cadru mai adecvat pentru analiza datelor EEG comparativ cu metodele tradiționale, oferind astfel un cadru mai robust pentru analiza și interpretarea activității neuronale.

2.1.1 Benzile de frecvență EEG

Semnalul EEG conține componente oscilatorii distribuite pe mai multe benzi de frecvență, fiecare fiind asociată cu anumite stări funcționale ale creierului. Analiza spectrală a acestor benzi este esențială pentru extragerea de caracteristici relevante și pentru înțelegerea activității neuronale. În Tabelul 2.1 sunt prezentate principalele benzi de frecvență utilizate în analiza semnalelor EEG.

Tabela 2.1: Principalele benzi de frecvență ale semnalului EEG și asocierile funcționale ale acestora.

Bandă	Interval (Hz)	Asociere funcțională
Delta (δ)	0.5 – 4	Somn profund, procese regenerative; amplitudine crescută poate indica leziuni cerebrale
Theta (θ)	4 – 8	Somnolență, meditație, procesare memoriei; creșterea activității theta poate fi asociată cu declinul cognitiv
Alpha (α)	8 – 12	Stare de relaxare cu ochii închiși, repaus mental; reducerea puterii alpha este frecvent asociată cu boala Alzheimer
Beta (β)	12 – 30	Concentrare activă, gândire analitică, activitate motorie

În contextul afecțiunilor neurodegenerative, modificările în puterea spectrală a acestor benzi constituie biomarkeri importanți. De exemplu, pacienții cu boala Alzheimer prezintă frecvent o scădere a activității alpha și o creștere a activității theta și delta comparativ cu subiecții sănătoși [2]. Aceste diferențe spectrale motivează utilizarea puterii pe benzi de frecvență ca trăsături ale nodurilor în reprezentarea de tip graf a datelor EEG, abordare adoptată și în cadrul prezentei lucrări.

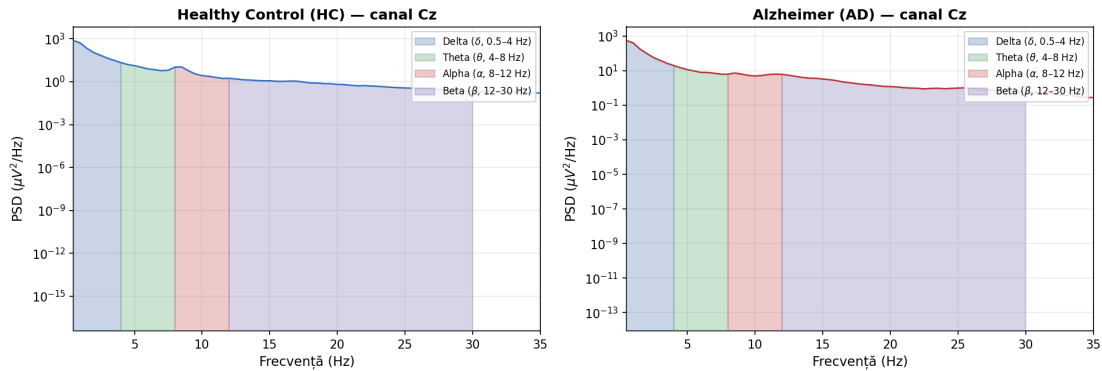


Figura 2.2: Înregistrări ale semnalelor EEG pe principalele benzi de frecvență (Delta 0.5–4 Hz, Theta 4–8 Hz, Alpha 8–12 Hz, Beta 12–30 Hz) pentru un subiect sănătos (HC) și un pacient cu Alzheimer (AD).

2.2 Tehnici de pre-procesare

Semnalul EEG brut nu este potrivit pentru analiza directă, fiind caracterizat de prezența zgomotului și a artefactelor. Înainte de aplicarea metodelor de învățare automată, este necesară efectuarea unor pași de pre-procesare, cu scopul de a îmbunătăți calitatea datelor și de a evidenția componentele relevante ale semnalului. Calitatea acestor etape influențează în mod direct performanța modelelor utilizate ulterior.

În cadrul unei înregistrări de electroencefalografie, semnalul poate prezenta diverse artefacte provenite din mișcări oculare (EOG), activitate musculară (EMG) sau interferențe electromagnetice. Aceste artefacte pot distorsiona semnalul cerebral real și

pot conduce la interpretări eronate dacă nu sunt tratate corespunzător. De asemenea, semnalele EEG sunt afectate de zgomot de fond, generat atât de echipamentele de înregistrare, cât și de mediul înconjurător.

Un pas esențial în pre-procesarea datelor EEG îl reprezintă filtrarea semnalului. Prin aplicarea unui filtru trece-bandă (band-pass filter), sunt păstrate doar frecvențele relevante pentru analiza activității cerebrale, în timp ce componentele de frecvență foarte joasă sau foarte înaltă, asociate cu zgomotul și artefactele, sunt eliminate.

Cel mai frecvent utilizat tip de filtru în analiza semnalelor EEG este filtrul FIR (Finite Impulse Response), care prezintă avantajul unei faze liniare, ceea ce înseamnă că nu introduce distorsiuni de fază în semnalul filtrat. În mod tipic, se aplică un filtru trece-bandă cu frecvența inferioară de 0.5 Hz (pentru eliminarea componentelor de curent continuu și a derivatei lente) și frecvența superioară de 45 Hz (pentru eliminarea zgomotului de rețea la 50 Hz și a altor interferențe de înaltă frecvență) [3, 4].

Pe lângă filtrare, pot fi utilizate tehnici avansate de reducere a artefactelor, precum Independent Component Analysis (ICA). Aceasta este o metodă de separare a semnalului în componente statistice independente, permițând identificarea și eliminarea componentelor asociate artefactelor (de exemplu, clipitul, mișcările oculare sau activitatea musculară), fără a afecta semnificativ componentele cerebrale utile [3, 4]. Detalii privind implementarea acestei etape sunt prezentate în Secțiunea 3.2.3.

După etapa de filtrare și curățare, semnalele EEG sunt adesea normalizate, pentru a asigura o scalare uniformă a datelor și pentru a facilita procesul de învățare al modelelor.

În continuare, semnalul este segmentat în ferestre de timp de lungime fixă, proces cunoscut sub numele de *epoching*. Fiecare segment (epocă) este tratat ca o instanță individuală, ceea ce permite captarea dinamicii temporale a activității cerebrale și crește numărul de exemple disponibile pentru antrenare. Această etapă este esențială în contextul abordărilor bazate pe grafuri, deoarece fiecare epocă va fi ulterior transformată într-un graf individual, în care nodurile corespund electrozilor, iar muchiile sunt definite pe baza relațiilor calculate pe acea fereastră de timp.

În concluzie, pre-procesarea reprezintă o etapă esențială în analiza datelor EEG, contribuind semnificativ la îmbunătățirea calității datelor și la creșterea performanței modelelor de învățare automată.

2.3 Conectivitate funcțională și Phase Locking Value

Un aspect central în construirea reprezentărilor de tip graf pentru datele EEG îl constituie definirea relațiilor dintre electrozi, adică a muchiilor grafului. În acest scop, se utilizează măsuri de conectivitate funcțională, care cuantifică gradul de sincronizare sau corelație dintre semnalele înregistrate de diferiți electrozi.

Dintre multiplele măsuri de conectivitate funcțională propuse în literatura de specialitate, Phase Locking Value (PLV) este una dintre cele mai utilizate în analiza semnalelor EEG [2]. PLV cuantifică consistența diferenței de fază dintre două semnale pe o perioadă de timp, oferind o măsură a sincronizării de fază care este independentă de amplitudine.

2.3.1 Transformata Hilbert și faza instantanee

Pentru a calcula PLV, este necesară mai întâi extragerea fazei instantanee a fiecărui semnal EEG. Aceasta se realizează prin intermediul transformatei Hilbert, care asociază unui semnal real $x(t)$ un semnal analitic complex:

$$z(t) = x(t) + i \cdot \mathcal{H}[x(t)] \quad (2.1)$$

unde:

- $x(t)$ – semnalul EEG real, în funcție de timp t ;
- $\mathcal{H}[x(t)]$ – transformata Hilbert a semnalului $x(t)$, care reprezintă componenta în cuadratură a acestuia;
- i – unitatea imaginară ($i^2 = -1$);
- $z(t)$ – semnalul analitic complex asociat.

Faza instantanee $\phi(t)$ este obținută ca argumentul semnalului analitic:

$$\phi(t) = \arg(z(t)) = \arctan\left(\frac{\mathcal{H}[x(t)]}{x(t)}\right) \quad (2.2)$$

unde $\arg(\cdot)$ desemnează argumentul (faza) unui număr complex, iar $\arctan(\cdot)$ este arcustangenta, calculată în practică prin funcția `atan2` pentru a obține valorile în intervalul $(-\pi, \pi]$.

Înainte de aplicarea transformatei Hilbert, semnalul este de regulă filtrat într-o bandă de frecvență specifică (de exemplu, banda Alpha, 8–12 Hz), pentru a obține o estimare stabilă și semnificativă din punct de vedere fiziologic a fazei instantanee.

2.3.2 Calculul PLV

Pentru două canale EEG i și j , Phase Locking Value este definit ca:

$$PLV_{ij} = \frac{1}{T} \left| \sum_{t=1}^T e^{i(\phi_i(t) - \phi_j(t))} \right| \quad (2.3)$$

unde:

- $\phi_i(t)$ – faza instantanee a canalului i la momentul t ;
- $\phi_j(t)$ – faza instantanee a canalului j la momentul t ;
- T – numărul total de eșantioane temporale ale epocii;
- $e^{i(\cdot)}$ – exponențiala complexă, care proiectează diferența de fază pe cercul unitate;
- $|\cdot|$ – modulul unui număr complex;
- $PLV_{ij} \in [0, 1]$ – valoarea de blocare a fazei dintre canalele i și j .

Valorile PLV sunt cuprinse în intervalul $[0, 1]$, unde:

- $PLV_{ij} = 0$ indică absența sincronizării de fază (diferența de fază variază aleatoriu în timp);
- $PLV_{ij} = 1$ indică o sincronizare perfectă de fază (diferența de fază este constantă pe întreaga perioadă de observare).

Prin calculul PLV pentru toate perechile de canale, se obține o matrice de conectivitate funcțională de dimensiune $N \times N$ (unde N este numărul de electrozi), care poate fi interpretată ca o matrice de adiacență ponderată a unui graf. Aplicarea unui prag (threshold) asupra acestei matrice permite eliminarea conexiunilor slabe și reținerea doar a relațiilor semnificative din punct de vedere funcțional [2, 4].

În cadrul implementării prezente, calculul PLV este aplicat în mod specific **benzii Alpha (8–12 Hz)**, motivat de relevanța clinică a acestei benzi în detectarea afecțiunilor neurodegenerative: reducerea activității alpha și modificarea sincronizării inter-regionale în această bandă constituie unul dintre biomarkerii consacrați ai bolii Alzheimer [2]. Înainte de extragerea fazei instantanee, fiecare semnal este filtrat în banda Alpha printr-un filtru Butterworth bidirecțional de ordinul 4.

Utilizarea PLV ca bază pentru construirea grafurilor EEG este motivată de capacitatea acestei măsuri de a captura sincronizarea funcțională dintre regiuni cerebrale, oferind o reprezentare a conectivității care reflectă dinamica reală a procesării neuronale, independent de amplitudinea semnalelor.

2.4 Rețelele neuronale de tip graf

În numeroase aplicații din lumea reală, datele nu pot fi reprezentate eficient prin structuri regulate de tip grilă, precum cele utilizate în cazul imaginilor. În schimb, acestea prezintă relații complexe și interdependențe care pot fi modelate mai natural sub forma unor grafuri. Într-un graf, entitățile sunt reprezentate prin noduri, iar relațiile dintre acestea prin muchii, permițând astfel descrierea unor structuri de date non-euclidiene.

În contextul analizei datelor EEG, această abordare devine deosebit de relevantă, deoarece electrozii pot fi interpretați ca noduri, iar conexiunile dintre diferite regiuni cerebrale pot fi modelate prin muchii. Această reprezentare permite captarea atât a relațiilor spațiale, cât și a celor funcționale dintre diferite zone ale creierului, depășind limitările impuse de metodele tradiționale bazate pe structuri regulate.

Rețelele neuronale de tip graf (Graph Neural Networks - GNN) reprezintă o extensie a modelelor de învățare profundă, concepute pentru a opera direct pe structuri de tip graf [2]. Acestea permit propagarea informației între noduri și învățarea unor reprezentări care integrează atât caracteristicile locale, cât și relațiile dintre noduri.

Datorită capacității lor de a modela date complexe și neregulate, rețelele neuronale graf au devenit o soluție promițătoare în numeroase domenii, inclusiv în analiza datelor biomedicale [4]. În continuare, vor fi prezentate principiile de funcționare ale acestor modele, precum și principalele tipuri de arhitecturi utilizate în practică.

2.4.1 Particularități ale GNN și mecanismul de message passing

Rețelele neuronale de tip graf sunt concepute pentru a învăța reprezentări ale datelor structurate sub formă de graf, prin integrarea informațiilor provenite atât din

caracteristicile nodurilor, cât și din relațiile dintre acestea. Spre deosebire de rețelele neuronale tradiționale, care operează pe structuri regulate, GNN-urile sunt capabile să proceseze direct grafuri de dimensiuni și forme variabile [4].

Conceptul fundamental care stă la baza funcționării GNN este mecanismul de propagare a informației între noduri, cunoscut sub denumirea de *message passing*. Formal, fie $G = (V, E)$ un graf, unde V reprezintă mulțimea nodurilor, iar E mulțimea muchiilor. Fiecare nod $v \in V$ are asociat un vector de caracteristici $\mathbf{h}_v^{(l)}$ la stratul l . Procesul de actualizare al reprezentărilor la stratul $l + 1$ poate fi descris prin două etape:

1. Agregare: Informația de la nodurile vecine este colectată printr-o funcție de agregare:

$$\mathbf{m}_v^{(l+1)} = \text{AGGREGATE}^{(l)}(\{\mathbf{h}_u^{(l)} : u \in \mathcal{N}(v)\}) \quad (2.4)$$

unde:

- $\mathbf{m}_v^{(l+1)}$ – mesajul agregat primit de nodul v de la vecinii săi, la trecerea spre stratul $l + 1$;
- $\mathcal{N}(v)$ – mulțimea vecinilor nodului v în graf;
- $\mathbf{h}_u^{(l)}$ – reprezentarea nodului vecin u la stratul curent l ;
- $\text{AGGREGATE}^{(l)}(\cdot)$ – funcție de agregare (e.g., sumă, medie sau operație mai complexă) specifică stratului l .

2. Actualizare: Reprezentarea nodului este actualizată prin combinarea informațiilor agregate cu reprezentarea curentă:

$$\mathbf{h}_v^{(l+1)} = \text{UPDATE}^{(l)}(\mathbf{h}_v^{(l)}, \mathbf{m}_v^{(l+1)}) \quad (2.5)$$

unde:

- $\mathbf{h}_v^{(l+1)}$ – noua reprezentare a nodului v la stratul $l + 1$;
- $\mathbf{h}_v^{(l)}$ – reprezentarea curentă a nodului v la stratul l ;
- $\mathbf{m}_v^{(l+1)}$ – mesajul agregat calculat în etapa anterioară;
- $\text{UPDATE}^{(l)}(\cdot)$ – funcție de actualizare care implică, de regulă, o transformare liniară urmată de o funcție de activare neliniară.

Prin repetarea acestor pași pe mai multe straturi, fiecare nod ajunge să integreze informații dintr-o vecinătate tot mai extinsă, permițând astfel modelarea relațiilor complexe dintre noduri [2]. Acest lucru este deosebit de util în cazul datelor EEG, unde interdependențele dintre diferitele regiuni ale creierului pot fi captate eficient prin această abordare.

Un alt aspect important al rețelelor neuronale graf este capacitatea acestora de a generaliza pe grafuri diferite, chiar dacă acestea au dimensiuni sau structuri variate. Această proprietate le face potrivite pentru aplicații în care datele sunt neregulate și dinamice, cum este cazul semnalelor EEG.

2.4.2 Graph Convolutional Networks (GCN)

Graph Convolutional Networks (GCN) reprezintă una dintre cele mai fundamentale și utilizate arhitecturi de rețele neuronale graf. Acestea extind operația de convoluție din domeniul imaginilor la structuri de tip graf, permițând agregarea informației de la nodurile vecine.

Regula de propagare a unui strat GCN, introdusă de Kipf și Welling [6], este definită astfel:

$$H^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)} \right) \quad (2.6)$$

unde:

- $H^{(l)} \in \mathbb{R}^{N \times d_l}$ – matricea reprezentărilor nodurilor la stratul l , cu N numărul de noduri și d_l dimensiunea reprezentărilor;
- $\tilde{A} = A + I_N$ – matricea de adiacență cu auto-conexiuni adăugate, unde A este matricea de adiacență originală, iar I_N este matricea identitate de dimensiune N ;
- $\tilde{D}$ – matricea diagonală de grade asociată lui $\tilde{A}$, cu $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$;
- $W^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$ – matricea de ponderi antrenabilă a stratului l ;
- $\sigma(\cdot)$ – funcție de activare neliniară (de exemplu, ReLU).

Termenul $\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}}$ realizează o normalizare simetrică a matricei de adiacență, ceea ce previne instabilitatea numerică și asigură o scalare adecvată a informației agregate de la vecini.

În implementarea prezentă, modelul GCN utilizează **trei straturi de convoluție**, funcția de activare **ELU** (Exponential Linear Unit) în loc de ReLU, și **conexiuni reziduale** (*skip connections*) la straturile 2 și 3, pentru a facilita propagarea gradientului și a preveni degradarea performanței la adâncimi mai mari. De asemenea, în loc de o singură operație de agregare globală, se utilizează o strategie de **dual pooling**: reprezentările nodurilor sunt agregate atât prin medie globală (*global mean pooling*), cât și prin maximum global (*global max pooling*), iar cele două rezultate sunt concatenate, producând un vector de lungime $2H$ care captează atât trăsăturile medii, cât și cele mai proeminente ale grafului.

O limitare teoretică a GCN rămâne aceea că ponderile muchiilor contribuie uniform la agregare (ponderate de gradul nodului), fără a diferenția importanța relativă a conexiunilor. Această abordare poate reduce capacitatea modelului de a surprinde relații complexe sau heterogene între noduri [4], limitare depășită de arhitectura GAT descrisă în subsecțiunea următoare.

2.4.3 Graph Attention Networks (GAT)

Pentru a depăși limitările modelelor de tip GCN, a fost introdusă arhitectura Graph Attention Networks (GAT), care utilizează mecanisme de atenție pentru a învăța importanța relativă a fiecărui vecin în procesul de agregare.

Spre deosebire de GCN, unde contribuția fiecărui nod vecin este determinată exclusiv de structura grafului, GAT atribuie coeficienți de atenție diferiți fiecărei muchii, permițând modelului să acorde o importanță mai mare nodurilor relevante și să ignore conexiunile mai puțin semnificative.

Coefficienții de atenție sunt calculați astfel. Mai întâi, se determină un scor de atenție nenormalizat e_{ij} pentru fiecare pereche de noduri vecine i și j :

$$e_{ij} = \text{LeakyReLU}(\mathbf{a}^T [W\mathbf{h}_i \| W\mathbf{h}_j]) \quad (2.7)$$

unde:

- $W \in \mathbb{R}^{d' \times d}$ – matricea de transformare liniară aplicată reprezentărilor nodurilor;
- $\mathbf{h}_i, \mathbf{h}_j \in \mathbb{R}^d$ – reprezentările nodurilor i , respectiv j , la stratul curent;
- $\mathbf{a} \in \mathbb{R}^{2d'}$ – vectorul de atenție antrenabil;
- $\|$ – operația de concatenare a doi vectori;
- $\text{LeakyReLU}(\cdot)$ – funcția de activare Leaky ReLU, cu pantă negativă mică pentru valori sub zero.

Acești coeficienți sunt apoi normalizați prin softmax peste vecinătatea nodului i :

$$\alpha_{ij} = \text{softmax}_j(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})} \quad (2.8)$$

unde:

- $\alpha_{ij} \in [0, 1]$ – coeficientul de atenție normalizat asociat muchiei de la nodul j la nodul i ;
- $\mathcal{N}(i)$ – mulțimea vecinilor nodului i ;
- normalizarea prin softmax asigură că $\sum_{k \in \mathcal{N}(i)} \alpha_{ik} = 1$.

Reprezentarea actualizată a nodului i se obține prin agregarea ponderată a reprezentărilor vecinilor:

$$\mathbf{h}_i^{(l+1)} = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} W \mathbf{h}_j^{(l)} \right) \quad (2.9)$$

unde $\sigma(\cdot)$ este o funcție de activare neliniară, iar ceilalți termeni au semnificațiile definite anterior.

În practică, GAT utilizează mecanismul de atenție multi-head, în care K funcții de atenție independente sunt aplicate în paralel, iar rezultatele sunt concatenate (în straturile intermediare) sau mediate (în stratul final):

$$\mathbf{h}_i^{(l+1)} = \left\|_{k=1}^K \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} W^{(k)} \mathbf{h}_j^{(l)} \right) \right. \quad (2.10)$$

unde:

- K – numărul de capuri de atenție independente;
- $\alpha_{ij}^{(k)}$ – coeficientul de atenție al capului k pentru muchia (i, j) ;
- $W^{(k)}$ – matricea de transformare liniară asociată capului de atenție k ;

- $\parallel$ – operația de concatenare a rezultatelor celor K capuri.

În implementarea prezentă, modelul GAT utilizează de asemenea **trei straturi de atenție** și o strategie de **dual pooling** identică celei descrise pentru GCN. Un element arhitectural distinctiv este prezența unei **conexiuni reziduale de tip skip** de la output-ul primului strat la output-ul celui de-al treilea strat. Deoarece dimensiunile celor două reprezentări diferă (primul strat produce un vector de dimensiune $K_1 \cdot H$, iar al treilea strat un vector de dimensiune H), conexiunea reziduală este mediată printr-o **proiecție liniară** $W_{proj} \in \mathbb{R}^{H \times K_1 H}$:

$$\mathbf{h}_i^{(3)} = \text{ELU}\left(\text{GAT}^{(3)}\left(\mathbf{h}_i^{(2)}\right)\right) + W_{proj} \mathbf{h}_i^{(1)} \quad (2.11)$$

Această formulare permite reutilizarea directă a reprezentărilor de la primul nivel de atenție, îmbunătățind fluxul gradientului în timpul antrenării.

Utilizarea mecanismului de atenție permite o modelare mai flexibilă și adaptivă a relațiilor din graf, fiind deosebit de utilă în situațiile în care conexiunile dintre noduri nu au aceeași importanță. Totodată, GAT nu necesită cunoașterea explicită a structurii globale a grafului, ceea ce îl face potrivit pentru aplicații pe date complexe și neregulate, cum sunt semnalele EEG [4].

2.4.4 Relevanța arhitecturilor GCN și GAT pentru analiza EEG

În contextul analizei datelor EEG, alegerea arhitecturii potrivite este esențială pentru captarea relațiilor funcționale dintre diferitele regiuni cerebrale. Modelele de tip GCN oferă o abordare simplă și eficientă pentru agregarea informației, fiind potrivite în situațiile în care conexiunile dintre noduri pot fi considerate relativ uniforme.

Pe de altă parte, modelele GAT permit diferențierea importanței conexiunilor dintre electrozi, ceea ce este deosebit de relevant în analiza semnalelor EEG, unde anumite regiuni cerebrale pot avea o influență mai mare decât altele în funcție de sarcina analizată. Astfel, GAT oferă un cadru mai flexibil pentru modelarea relațiilor complexe și dinamice caracteristice datelor EEG.

Din aceste considerente, ambele tipuri de arhitecturi sunt frecvent utilizate în literatura de specialitate pentru procesarea semnalelor EEG, fiind investigate în funcție de specificul aplicației și de natura datelor disponibile [2, 4]. În cadrul prezentei lucrări, vor fi implementate și evaluate comparativ ambele arhitecturi, pentru a analiza performanța lor în clasificarea semnalelor EEG asociate afecțiunilor neurodegenerative.

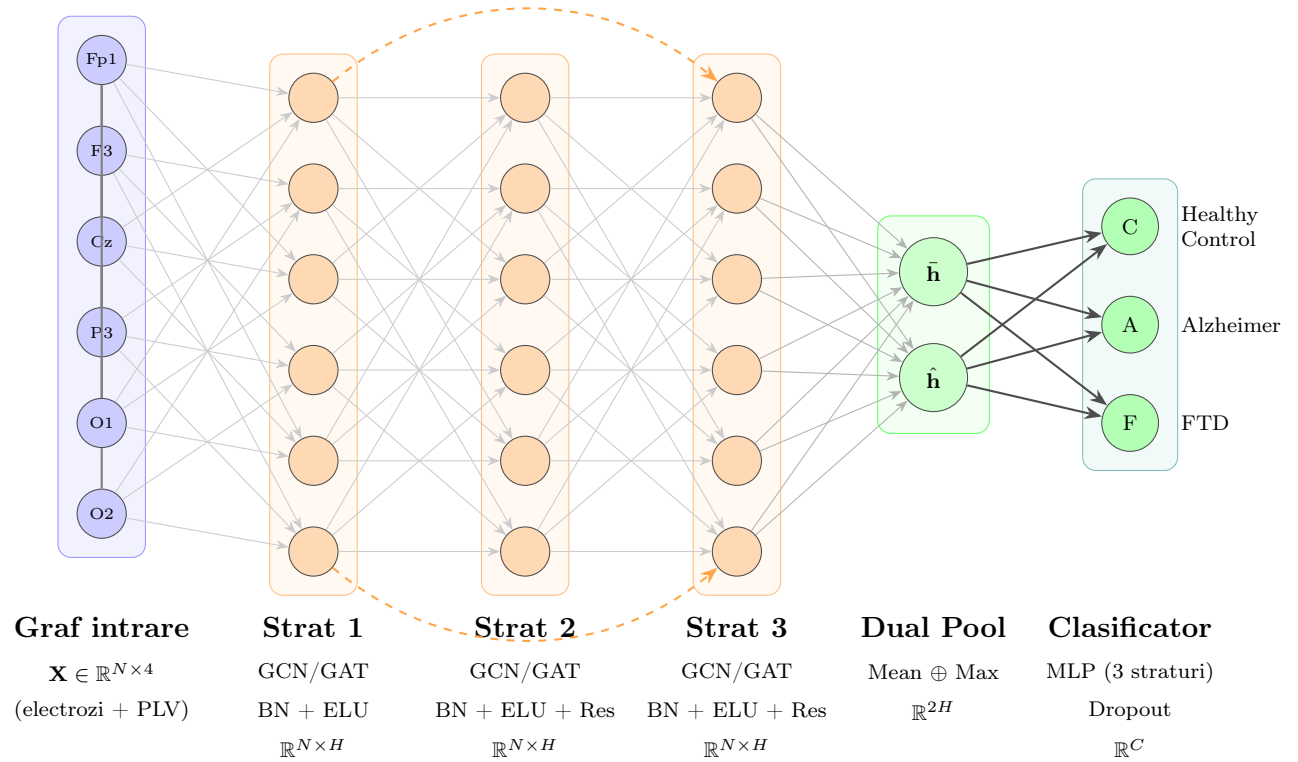


Figura 2.3: Arhitectura rețelei neuronale de tip graf (GCN/GAT) utilizate pentru clasificarea semnalelor EEG: grafurile de intrare cu electrozi și conectivitate ponderată, trei straturi de propagare cu normalizare batch, activare ELU și conexiuni reziduale, agregare globală prin concatenarea mediei și maximului (*dual pooling*), și clasificador MLP cu trei straturi liniare. Săgețile portocalii punctate reprezintă conexiunile reziduale (*skip connections*).

2.5 Procesarea semnalelor EEG utilizând rețele neuronale de tip graf

În ultimii ani, utilizarea rețelelor neuronale de tip graf (Graph Neural Networks - GNN) în analiza semnalelor EEG a cunoscut o creștere semnificativă, datorită capacității acestora de a modela relațiile complexe dintre diferitele regiuni ale creierului. Spre deosebire de metodele tradiționale, care tratează canalele EEG independent sau presupun relații locale fixe, abordările bazate pe grafuri permit captarea interdependenței funcționale și structurale dintre electrozi, oferind o reprezentare mai fidelă a activității cerebrale [2].

Un aspect central al acestor metode îl reprezintă modul în care este construit grafurile asociate datelor EEG. În majoritatea lucrărilor, electrozii sunt modelați ca noduri, iar muchiile sunt definite fie pe baza distanței fizice dintre electrozi, fie pe baza unor măsuri de conectivitate funcțională, precum corelația sau coerența semnalelor. Această flexibilitate permite adaptarea structurii grafurilor în funcție de aplicație și de natura datelor analizate [4].

Numeroase studii au demonstrat eficiența utilizării modelelor GNN în diverse sarcini legate de procesarea semnalelor EEG, precum clasificarea stărilor cognitive, detecția

crizelor epileptice sau identificarea afecțiunilor neurologice și neurodegenerative. De exemplu, în contextul detecției epilepsiei, modelele GNN au fost utilizate pentru a surprinde tiparele spațio-temporale ale activității cerebrale, obținând performanțe superioare față de metodele clasice de învățare automată [3].

Un domeniu deosebit de important în care rețelele neuronale graf au demonstrat un potențial ridicat este diagnosticarea și analiza bolilor neurodegenerative, în special a bolii Alzheimer. Această afecțiune este caracterizată prin modificări subtile ale activității cerebrale, care pot fi dificil de detectat prin metode tradiționale. Utilizarea GNN permite modelarea rețelelor funcționale ale creierului și identificarea unor tipare specifice asociate declinului cognitiv.

În acest context, studiul realizat de Hu et al. propune un model de rețea neuronală graf explicabilă pentru predicția riscului asociat bolii Alzheimer și dementelor înrudite, demonstrând potențialul acestor modele în aplicații biomedicale cu componentă de interpretabilitate [1]. Această caracteristică este deosebit de importantă în aplicațiile medicale, unde încrederea în model și capacitatea de a explica deciziile sunt esențiale pentru adoptarea pe scară largă a acestor tehnologii.

Pe lângă modelele GCN, care oferă o agregare eficientă a informației, lucrările recente evidențiază avantajele utilizării mecanismelor de atenție, precum în cazul modelelor GAT. Acestea permit identificarea conexiunilor relevante dintre regiuni cerebrale și adaptarea dinamică a ponderilor asociate muchiilor, ceea ce este esențial în analiza datelor EEG, unde relațiile dintre canale pot varia în timp și în funcție de sarcina analizată [2, 4].

În general, abordările bazate pe GNN oferă un cadru unificat pentru integrarea informațiilor spațiale și temporale din semnalele EEG. Prin modelarea explicită a relațiilor dintre electrozi, aceste metode reușesc să captureze structura complexă a activității cerebrale și să îmbunătățească performanța în sarcini de clasificare și detecție.

Cu toate acestea, există și provocări asociate utilizării GNN în acest domeniu. Printre acestea se numără alegerea optimă a structurii grafului, scalabilitatea modelelor și necesitatea unor seturi de date bine etichetate. În plus, variabilitatea ridicată a semnalelor EEG între subiecți poate afecta capacitatea de generalizare a modelelor.

În concluzie, rețelele neuronale de tip graf reprezintă o direcție promițătoare în analiza semnalelor EEG, oferind avantaje semnificative față de metodele tradiționale. Capacitatea acestora de a modela relațiile complexe dintre regiunile cerebrale le face deosebit de potrivite pentru aplicații în domeniul biomedical, inclusiv pentru detectarea și monitorizarea bolilor neurodegenerative precum Alzheimer.

Capitolul 3

Implementare

În cadrul acestui capitol este descris în detaliu procesul de implementare al întregului pipeline propus pentru analiza și clasificarea semnalelor EEG utilizând rețele neuronale de tip graf. Sunt prezentate setul de date utilizat, etapele concrete de pre-procesare, construirea reprezentărilor de tip graf, arhitecturile modelelor implementate, procesul de antrenare și mecanismul de inferență. Implementarea a fost realizată în limbajul Python, utilizând bibliotecile MNE [7] pentru procesarea semnalelor EEG și PyTorch Geometric [5] pentru construirea și antrenarea modelelor de rețele neuronale graf.

3.1 Setul de date utilizat

Setul de date utilizat în cadrul prezentei lucrări conține înregistrări EEG provenite de la subiecți din trei categorii clinice: persoane sănătoase (Healthy Control – C), pacienți diagnosticați cu boala Alzheimer (A) și pacienți diagnosticați cu demență frontotemporală (F).¹ Înregistrările corespund protocolului experimental *eyes closed*, în care subiecții sunt monitorizați în stare de repaus cu ochii închiși, protocol frecvent utilizat în studiile privind afecțiunile neurodegenerative.

3.1.1 Formatul și structura datelor

Fișierele EEG sunt stocate în formatul EEGLAB `.set`, un format standard utilizat pe scară largă în comunitatea de cercetare în neuroștiințe. Structura directoarelor urmează convenția BIDS (Brain Imaging Data Structure), fiecare subiect având un director dedicat care conține înregistrarea EEG corespunzătoare:

```
data/
|-- participants_train.tsv
|-- derivatives/
|   |-- sub-001/
|   |   +-- eeg/
|   |       +-- sub-001_task-eyesclosed_eeg.set
|   |-- sub-002/
|   |   +-- eeg/
|   |       +-- sub-002_task-eyesclosed_eeg.set
```

¹Setul de date este disponibil public la adresa: <https://openneuro.org/datasets/ds004504>.

```
|   +-- ...
+-- holdout_diagnosis/
|   |-- sub-024/
|       +-- eeg/
|           +-- sub-024_task-eyesclosed_eeg.set
+-- ...
```

Fișierul `participants_train.tsv` conține metadatele asociate fiecărui subiect, incluzând identificatorul unic (`participant_id`) și grupul clinic (`Group`). Acest fișier este utilizat pentru a asocia fiecărei înregistrări EEG eticheta corespunzătoare în procesul de construire a setului de date pentru antrenare.

Directorul `holdout_diagnosis/` conține înregistrări EEG rezervate exclusiv pentru etapa de inferență și testare finală, aceste date nefiind utilizate în procesul de antrenare sau validare a modelelor.

3.1.2 Configurația claselor de clasificare

Sistemul implementat suportă două moduri de clasificare, configurabile de către utilizator:

- **Clasificare binară** – în care sunt utilizate doar grupurile C (Healthy Control) și A (Alzheimer’s Disease), subiecții din grupul F fiind excluși. Etichetele asociate sunt: 0 pentru clasa C și 1 pentru clasa A.
- **Clasificare pe trei clase** – în care sunt utilizate toate cele trei grupuri: C, A și F (Frontotemporal Dementia). Etichetele asociate sunt: 0 pentru C, 1 pentru A și 2 pentru F.

Această flexibilitate permite evaluarea modelelor atât în scenariul simplificat de diferențiere între starea sănătoasă și boala Alzheimer, cât și în scenariul mai complex de clasificare a trei tipuri distincte de stări cognitive. Maparea dintre grupurile clinice și etichetele numerice este centralizată într-un dicționar de configurare, asigurând consistența pe tot parcursul pipeline-ului.

3.1.3 Selecția canalelor EEG

Din totalul canalelor disponibile în fiecare înregistrare, sunt reținute doar cele 19 canale corespunzătoare sistemului internațional de plasare a electrozilor 10-20, un standard utilizat pe scară largă în electroencefalografia clinică și de cercetare. Canalele selectate sunt: Fp1, Fp2, F7, F3, Fz, F4, F8, T3, C3, Cz, C4, T4, T5, P3, Pz, P4, T6, O1 și O2.

Această selecție asigură o acoperire uniformă a regiunilor cerebrale (frontală, centrală, temporală, parietală și occipitală) și garantează compatibilitatea între înregistrările provenite de la diferiți subiecți, eliminând eventualele canale suplimentare sau specifice anumitor configurații de echipamente.

3.2 Pre-procesarea semnalelor EEG

Etapa de pre-procesare reprezintă primul pas operațional în pipeline-ul implementat și are rolul de a transforma înregistrările EEG brute în date curate și structurate, potrivite pentru extragerea caracteristicilor și construirea ulterioară a reprezentărilor de tip graf. Întreaga logică de pre-procesare este implementată în modulul `src/preprocessing.py`, utilizând biblioteca MNE [7]. Pașii principali ai acestui modul sunt ilustrați în ordinea lor de execuție: încărcarea fișierului, selecția canalelor, filtrarea, curățarea opțională prin ICA, segmentarea în ferestre de timp și conversia la format NumPy.

3.2.1 Încărcarea fișierelor EEG

Fișierele EEG în format EEGLAB `.set` sunt încărcate prin intermediul funcției `mne.io.read_raw_eeglab()` din biblioteca MNE, cu parametrul `preload=True`, care determină încărcarea completă a datelor în memoria RAM în momentul citirii. Rezultatul este un obiect de tip `mne.io.BaseRaw`, care încapsulează atât datele brute ale semnalului, cât și metadatele asociate înregistrării: rata de eșantionare f_s , lista denumirilor canalelor, durata totală a înregistrării și informații despre sistemul de achiziție.

Verificarea existenței fizice a fișierului pe disc este realizată anterior apelului de citire, generând o excepție explicită (`FileNotFoundException`) în cazul în care fișierul nu poate fi localizat. Această validare preventivă asigură un comportament robust al pipeline-ului în cazul unor seturi de date incomplete sau al unor erori de configurare a căilor de acces.

3.2.2 Filtrarea trece-bandă

După încărcare și selecția canalelor standard (descrișă în Secțiunea 3.1.3), semnalul este supus unui proces de filtrare trece-bandă, cu scopul de a elimina componentele spectrale irelevante sau perturbatoare și de a păstra exclusiv banda de frecvență de interes pentru analiza activității cerebrale.

Se aplică un filtru FIR (Finite Impulse Response) de tip *firwin* (fereastră Hanning), cu frecvența inferioară $f_l = 0.5$ Hz și frecvența superioară $f_h = 45.0$ Hz, prin intermediul metodei `raw.filter(l_freq, h_freq, fir_design="firwin")` din biblioteca MNE.

Alegerea acestor limite de frecvență este motivată fiziologic și tehnic:

- Frecvența inferioară $f_l = 0.5$ Hz elimină componenta de curent continuu (DC offset) și deriva lentă a liniei de bază (baseline drift), care pot apărea datorită mișcărilor subiectului sau impedanței variabile a electrozilor;
- Frecvența superioară $f_h = 45.0$ Hz elimină artefactele de înaltă frecvență și interferențele de la rețeaua electrică (50 Hz în Europa), fără a afecta benzile de interes clinic (Delta, Theta, Alpha, Beta – toate sub 30 Hz).

Filtrul FIR a fost ales față de alternativa IIR (Infinite Impulse Response) deoarece garantează o **fază liniară**: toate componentele frecvențiale ale semnalului sunt întârziate cu același interval de timp, fără distorsiuni de fază. Această proprietate este esențială în analiza semnalelor EEG, unde relațiile de fază dintre canale constituie baza calculului PLV descris în Secțiunea 2.3 [3, 4].

3.2.3 Curățarea prin ICA

Opțional, după filtrare, poate fi aplicat un pas de curățare a artefactelor prin Analiza Componentelor Independente (Independent Component Analysis – ICA). Această etapă este controlată prin parametrul boolean `apply_ica` al funcției principale de pre-procesare și este dezactivată implicit (`apply_ica=False`).

Algoritmul ICA descompune semnalul EEG multicanal într-un set de componente statistice independente. Formal, fie $\mathbf{X} \in \mathbb{R}^{N_c \times T}$ matricea semnalului EEG, cu N_c canale și T eșantioane temporale. ICA caută o matrice de mixare $\mathbf{M}$ și sursele independente $\mathbf{S}$, astfel încât:

$$\mathbf{X} = \mathbf{M} \cdot \mathbf{S} \quad (3.1)$$

unde:

- $\mathbf{X} \in \mathbb{R}^{N_c \times T}$ – matricea semnalului EEG observat;
- $\mathbf{M} \in \mathbb{R}^{N_c \times n}$ – matricea de mixare (mixing matrix), cu n numărul de componente independente;
- $\mathbf{S} \in \mathbb{R}^{n \times T}$ – matricea surselor independente estimate.

Componentele asociate artefactelor tipice (clipit, mișcări oculare, activitate musculară) au, în general, o formă caracteristică în spațiul surselor și pot fi identificate și eliminate, iar semnalul este reconstruit din componentele rămase. Numărul de componente ICA este setat la $n = \min(15, N_c)$, iar starea inițială a algoritmului este fixată prin `random_state = 42` pentru a asigura reproductibilitatea rezultatelor.

Etapă ICA este lăsată opțională deoarece, în absența unei inspecții manuale a componentelor, există riscul eliminării unor componente cerebrale relevante. Acest aspect este cu atât mai important în contextul seturilor de date de cercetare clinică, unde minimizarea pierderii de informație este prioritară [3, 4].

3.2.4 Segmentarea în epoci

Semnalul continuu, după filtrare și curățare, este segmentat în ferestre de timp de lungime fixă printr-un proces denumit *epoching*. Această etapă transformă înregistrarea continuă într-o colecție de segmente independente, fiecare urmând a fi procesat ca o instanță separată în pipeline-ul de construire a grafurilor.

În implementarea de față, fiecare epocă are durata $\Delta t = 5$ secunde, fără suprapunere (overlap = 0 s). Evenimentele de segmentare sunt generate automat cu funcția `mne.make_fixed_length_events()`, iar obiectul `mne.Epochs` este construit cu parametrul `detrend=1`, care aplică o eliminare liniară a tendinței (*linear detrending*) pentru fiecare epocă în parte.

Detrending-ul liniar elimină tendința de primă ordine din fiecare epocă: se ajustează un polinom de gradul 1 pe datele epocii și se scade din semnal, prevenind astfel influența variațiilor lente de nivel (low-frequency drift) asupra caracteristicilor spectrale extrase ulterior.

Numărul de epoci N_e obținute dintr-o înregistrare de durată totală D secunde este aproximat prin:

$$N_e = \left\lfloor \frac{D}{\Delta t} \right\rfloor \quad (3.2)$$

unde:

- N_e – numărul de epoci extrase;
- D – durata totală a înregistrării EEG (secunde);
- $\Delta t = 5$ s – durata unei epoci;
- $\lfloor \cdot \rfloor$ – partea întreagă inferioară (floor).

Fiecare epocă rezultată este tratată ca o instanță independentă de antrenare, ceea ce multiplică semnificativ numărul de exemple disponibile per subiect și contribuie la creșterea diversității datelor de antrenare. Această alegere este standard în literatura de specialitate pentru clasificarea semnalelor EEG în stare de repaus [2, 3].

3.2.5 Conversia la format NumPy

În ultima etapă a modului de pre-procesare, epocile sunt convertite dintr-un obiect `mne.Epochs` într-un array NumPy tridimensional, utilizând metoda `epochs.get_data()`. Array-ul rezultat are forma:

$$\mathbf{X} \in \mathbb{R}^{N_e \times N_c \times N_t} \quad (3.3)$$

unde:

- N_e – numărul de epoci extrase din înregistrare;
- $N_c = 19$ – numărul de canale EEG, conform selecției sistemului 10-20;
- $N_t = \Delta t \cdot f_s$ – numărul de eșantioane temporale per epocă, determinat de durata epocii Δt și rata de eșantionare f_s a înregistrării.

Valorile din array sunt exprimate în volți (unitatea nativă a obiectului MNE). Conversia la microvolți (μV) este realizată ulterior, în modulul de construire a grafurilor, prin înmulțire cu factorul 10^6 , facilitând scalarea numerică adecvată pentru calculul densității spectrale de putere.

Acest tensor tridimensional $\mathbf{X}$ constituie ieșirea modului de pre-procesare și intrarea directă pentru modulul de construire a grafurilor, descris în secțiunea următoare.

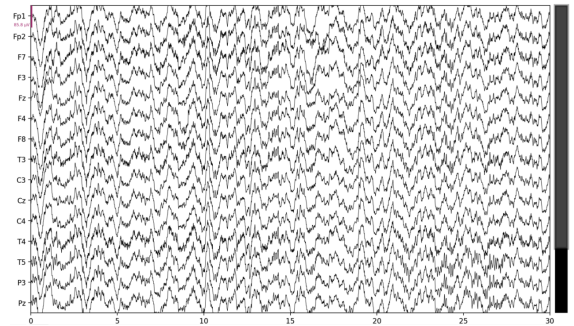
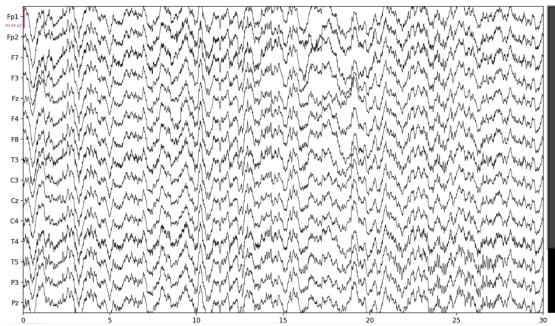


Figura 3.1: *
(a) Înainte de pre-procesare

Figura 3.2: *
(b) După pre-procesare

Figura 3.3: Semnalul EEG înainte și după etapa de pre-procesare.

3.3 Construirea reprezentărilor de tip graf

Odată ce semnalele EEG au fost pre-procesate și convertite în tensorul de epoci $\mathbf{X} \in \mathbb{R}^{N_e \times N_c \times N_t}$, fiecare epocă este transformată individual într-un graf compatibil cu biblioteca PyTorch Geometric. Această etapă este implementată în modulul `src/graph_builder.py` și cuprinde trei operații principale: extragerea trăsăturilor nodurilor, calculul matricei de conectivitate funcțională prin PLV și conversia la formatul intern al obiectului `Data` din PyG.

3.3.1 Extragerea trăsăturilor nodurilor

Fiecare nod al grafului corespunde unui electrod EEG și este caracterizat printr-un vector de trăsături care codifică informația spectrală a semnalului înregistrat de acel electrod în cadrul epocii respective. Trăsăturile sunt calculate ca puterea medie a semnalului pe cele patru benzi de frecvență standard: Delta (δ : 0.5–4 Hz), Theta (θ : 4–8 Hz), Alpha (α : 8–12 Hz) și Beta (β : 12–30 Hz), conform Tabelului 2.1.

Calculul puterii spectrale se realizează prin metoda Welch [3], care estimează densitatea spectrală de putere (PSD) prin medierea periodogramelor calculate pe segmente suprapuse ale semnalului. Pentru canalul c și banda de frecvență $[f_l, f_h]$, puterea medie este:

$$P_c^{[f_l, f_h]} = \frac{1}{|\mathcal{F}_{[f_l, f_h]}|} \sum_{f \in \mathcal{F}_{[f_l, f_h]}} \hat{S}_c(f) \quad (3.4)$$

unde:

- $P_c^{[f_l, f_h]}$ – puterea medie a canalului c în banda $[f_l, f_h]$;
- $\mathcal{F}_{[f_l, f_h]}$ – mulțimea bin-urilor de frecvență ale PSD care se încadrează în intervalul $[f_l, f_h]$;
- $\hat{S}_c(f)$ – estimatul PSD al canalului c la frecvența f , obținut prin metoda Welch;
- $|\mathcal{F}_{[f_l, f_h]}|$ – numărul de bin-uri de frecvență din bandă.

Lungimea segmentului utilizat de metoda Welch este $n_{seg} = \min(2f_s, N_t)$ eșantioane, unde f_s este rata de eșantionare și N_t este numărul de eșantioane ale epocii. Această alegere asigură o rezoluție spectrală de cel puțin 0.5 Hz, suficientă pentru delimitarea benzilor de interes. Datele sunt în prealabil convertite la microvolți (μV) prin înmulțire cu factorul 10^6 , facilitând o scalare numerică adecvată.

Rezultatul este matricea de trăsături ale nodurilor:

$$\mathbf{X}_{nod} \in \mathbb{R}^{N_c \times 4} \quad (3.5)$$

unde fiecare linie $c \in \{1, \dots, N_c\}$ conține vectorul $[P_c^\delta, P_c^\theta, P_c^\alpha, P_c^\beta]$ al puterii medii pe cele patru benzi. Cu $N_c = 19$ canale, fiecare nod este descris printr-un vector de dimensiune 4.

Alegerea puterii spectrale pe benzi ca trăsături ale nodurilor este motivată de relevanța fiziologică a acestor benzi în contextul afecțiunilor neurodegenerative, discutată în Secțiunea 2.1.1, și de caracterul lor compact și interpretabil, care facilitează atât antrenarea, cât și analiza ulterioară a modelului.

3.3.2 Calculul matricei de conectivitate funcțională

Muchiile grafului sunt definite pe baza sincronizării funcționale dintre perechi de electrozi, cuantificată prin Phase Locking Value (PLV), introdus teoretic în Secțiunea 2.3. Calculul PLV este aplicat în banda Alpha (8–12 Hz), motivat de relevanta acestei benzi în detectarea modificărilor asociate bolii Alzheimer, unde reducerea activității alpha constituie un biomarker clinic important [2].

Procesul de calcul pentru o epocă $\mathbf{e} \in \mathbb{R}^{N_c \times N_t}$ urmează trei pași:

Pasul 1 — Filtrare în bandă. Semnalul fiecărui canal este filtrat în banda Alpha printr-un filtru Butterworth trece-bandă de ordinul 4, implementat cu filtrare bidirecțională (`filtfilt`) pentru a elimina distorsiunile de fază:

$$\tilde{e}_c(t) = \text{filtfilt}_{[8,12] \text{ Hz}}(e_c(t)), \quad c = 1, \dots, N_c \quad (3.6)$$

Filtrarea bidirecțională aplică filtrul de două ori — o dată înainte și o dată înapoi în timp — rezultând o fază zero netă și o atenuare dublă față de un singur pas de filtrare.

Pasul 2 — Extragerea fazei instantanee. Se aplică transformata Hilbert semnalelor filtrate pentru a obține fazele instantanee, conform ecuațiilor (2.1) și (2.2):

$$\phi_c(t) = \arg(\tilde{e}_c(t) + i \cdot \mathcal{H}[\tilde{e}_c(t)]), \quad c = 1, \dots, N_c \quad (3.7)$$

Pasul 3 — Calculul PLV pentru toate perechile. PLV se calculează pentru fiecare pereche (i, j) cu $i < j$, conform ecuației (2.3), și se completează simetric:

$$\mathbf{PLV} \in \mathbb{R}^{N_c \times N_c}, \quad \mathbf{PLV}_{ij} = \mathbf{PLV}_{ji}, \quad \mathbf{PLV}_{ii} = 0 \quad (3.8)$$

Diagonala principală este setată la zero pentru a evita auto-conexiunile nodurilor. Matricea **PLV** constituie matricea de adiacență ponderată brută a grafului.

3.3.3 Aplicarea pragului de adiacență

Matricea PLV completă conține valori nenule pentru toate perechile de electrozi, ceea ce ar conduce la un graf dens, cu $N_c(N_c - 1)$ muchii direcționate, respectiv $\frac{N_c(N_c - 1)}{2}$ muchii nedirecționate. Un graf prea dens poate introduce zgomot structural și poate îngreuna procesul de învățare al modelului GNN, deoarece conexiunile cu sincronizare funcțională slabă nu transmit informație relevantă [4].

Pentru a obține un graf mai sparse și semnificativ din punct de vedere funcțional, se aplică un prag $\tau \in [0, 1]$ asupra matricei PLV:

$$\mathbf{A}_{ij} = \begin{cases} \mathbf{PLV}_{ij} & \text{dacă } \mathbf{PLV}_{ij} \geq \tau \\ 0 & \text{altfel} \end{cases} \quad (3.9)$$

unde:

- $\mathbf{A} \in \mathbb{R}^{N_c \times N_c}$ — matricea de adiacență ponderată a grafului, după aplicarea pragului;
- τ — pragul de conectivitate, parametru configurabil de către utilizator;
- valorile sub τ sunt setate la zero, eliminând muchiile cu sincronizare slabă.

În implementarea prezentă, valoarea implicită a pragului este $\tau = 0.9$, reținând astfel doar conexiunile cu un grad ridicat de sincronizare de fază. Alegerea unui prag ridicat este motivată de natura setului de date utilizat (înregistrări în repaus cu ochii închiși), în care conectivitatea funcțională relevantă clinic este, în general, caracterizată prin valori PLV ridicate [2]. Pragul este totuși un hiperparametru configurabil, al cărui impact asupra performanței modelului este analizat în Capitolul 5.

3.3.4 Conversia la formatul PyTorch Geometric

Matricea de adiacență $\mathbf{A}$ este convertită la reprezentarea internă utilizată de PyTorch Geometric, care stochează grafurile în format *sparse* prin două tensori:

- $\text{edge_index} \in \mathbb{Z}^{2 \times |\mathcal{E}|}$ – tensorii de indici ai muchiilor, unde prima linie conține indicii nodurilor sursă, iar a doua linie indicii nodurilor destinație; $|\mathcal{E}|$ este numărul total de muchii după pragare;
- $\text{edge_attr} \in \mathbb{R}^{|\mathcal{E}| \times 1}$ – tensorul atributelor muchiilor, conținând valoarea PLV corespunzătoare fiecărei muchii.

Conversia se realizează prin identificarea pozițiilor nenule din $\mathbf{A}$ cu `numpy.nonzero()`, extragerea valorilor corespunzătoare și construirea tensorilor PyTorch cu tipurile de date corespunzătoare (`torch.long` pentru indici și `torch.float32` pentru attribute).

3.3.5 Construirea obiectului Data

Toate componentele calculate sunt asamblate într-un obiect de tip **Data** din biblioteca `torch_geometric`, care reprezintă graful complet asociat unei epoci EEG:

$$\mathcal{G} = (\mathbf{X}_{nod}, \text{edge_index}, \text{edge_attr}, y, \text{subject_id}, \text{epoch_index}) \quad (3.10)$$

unde:

- $\mathbf{X}_{nod} \in \mathbb{R}^{19 \times 4}$ – matricea trăsăturilor nodurilor (puterea spectrală pe 4 benzi);
- $\text{edge_index} \in \mathbb{Z}^{2 \times |\mathcal{E}|}$ – conectivitatea grafului în format COO (Coordinate format);
- $\text{edge_attr} \in \mathbb{R}^{|\mathcal{E}| \times 1}$ – valorile PLV ale muchiilor;
- $y \in \{0, 1\}$ sau $y \in \{0, 1, 2\}$ – eticheta clinică a subiectului (Healthy Control, Alzheimer, Frontotemporal Dementia);
- subject_id – identificatorul subiectului de origine, utilizat pentru split-ul subiect-nivel la antrenare;
- epoch_index – indexul ferestrei temporale în cadrul înregistrării subiectului.

Prin parcurgerea tuturor ferestrelor temporale ale tuturor subiecților incluși în setul de date, se construiește o listă de obiecte **Data**, câte unul per fereastră, care constituie dataset-ul final utilizat la antrenare. Procesul de construire a acestui dataset este orchestrat de modulul `src/dataset_builder.py`, descris în secțiunea următoare.

3.4 Construirea setului de date

Modulul `src/dataset_builder.py` orchestrează întregul flux de construire a setului de date, de la citirea metadatelor clinice până la salvarea colecției finale de grafuri PyG pe disc. Acesta integrează secvențial operațiile descrise în Secțiunile 3.2 și 3.3, aplicându-le pentru fiecare subiect inclus în setul de antrenare.

3.4.1 Citirea metadatelor și maparea etichetelor

Punctul de intrare al procesului îl constituie fișierul `participants_train.tsv`, care este citit ca un tabel structurat cu ajutorul bibliotecii `pandas`. Fiecare rând al tabelului conține identificatorul unui subiect (`participant_id`) și grupul clinic din care face parte (`Group`).

Înainte de procesare, tabelul este validat pentru a asigura prezența coloanelor obligatorii. Grupul clinic al fiecărui subiect este normalizat la majuscule și comparat cu mulțimea grupurilor active pentru modul de clasificare selectat. Subiecții care aparțin unor grupuri excluse din configurația curentă sunt omisi fără a genera erori, iar motivul omiterii este jurnalizat explicit. Maparea dintre grupurile clinice și etichetele numerice utilizate la antrenare este centralizată în dicționarul de configurare prezentat în Tabelul 3.1.

Tabela 3.1: Maparea grupurilor clinice la etichete numerice în funcție de modul de clasificare.

Grup clinic	Etichetă (binar)	Etichetă (trei clase)
C – Healthy Control	0	0
A – Alzheimer’s Disease	1	1
F – Frontotemporal Dementia	exclus	2

3.4.2 Procesarea per subiect

Pentru fiecare subiect inclus, pipeline-ul execută secvențial următoarele operații:

1. **Localizarea fișierului EEG** – calea fișierului este construită după convenția BIDS, prezentată în Secțiunea 3.1.1. Dacă fișierul nu există pe disc, subiectul este omis și evenimentul este jurnalizat;
2. **Pre-procesarea semnalului** – se apelează funcția `load_and_preprocess_eeg()` din modulul `src/preprocessing.py`, care aplică secvența completă descrisă în Secțiunea 3.2: încărcare, selecție canale, filtrare FIR și, opțional, curățare ICA;
3. **Segmentarea în epoci** – semnalul pre-procesat este segmentat în epoci de $\Delta t = 5$ s fără suprapunere, obținând tensorul $\mathbf{X} \in \mathbb{R}^{N_e \times 19 \times N_t}$;
4. **Construirea grafurilor** – pentru fiecare dintre cele N_e epoci se construiește câte un obiect `Data` conform procedurii descrise în Secțiunea 3.3, utilizând pragul PLV configurat. Fiecare graf primește eticheta clinică a subiectului și identificatorul acestuia ca metadata;
5. **Agregarea grafurilor** – grafurile generate pentru subiectul curent sunt adăugate la lista globală a setului de date.

În cazul în care procesarea unui subiect eșuează din motive tehnice (fișier corupt, număr insuficient de canale etc.), eroarea este capturată, motivul este jurnalizat, iar pipeline-ul continuă cu următorul subiect, fără a întrerupe procesarea întregului set de date.

3.4.3 Raportarea statisticilor și salvarea pe disc

La finalizarea procesării tuturor subiecților, modulul afișează un raport sumar în terminal, care include numărul de subiecți incluși, omisi și eșuați, numărul total de grafuri generate și distribuția acestora pe grupuri clinice. Aceste informații permit verificarea integrității setului de date înainte de lansarea antrenării și oferă o imagine de ansamblu asupra echilibrului dintre clase.

Setul de date final, reprezentat ca o listă de obiecte `Data`, este serializat și salvat pe disc ca fișier binar `.pt` utilizând funcția `torch.save()`. Acest format permite reîncărcarea rapidă a setului de date la antrenări ulterioare, fără a relua procesarea EEG, prin apelul `torch.load()`. Structura unui element al setului de date salvat este sintetizată în Tabelul 3.2.

Tabela 3.2: Câmpurile unui obiect `Data` din setul de date construit.

Câmp	Dimensiune	Descriere
<code>x</code>	19×4	Trăsăturile nodurilor: puterea spectrală pe 4 benzi
<code>edge_index</code>	$2 \times \mathcal{E} $	Indicii muchiilor în format COO
<code>edge_attr</code>	$ \mathcal{E} \times 1$	Valorile PLV ale muchiilor
<code>y</code>	scalar	Eticheta clinică numerică a subiectului
<code>subject_id</code>	string	Identificatorul subiectului de origine
<code>epoch_index</code>	scalar	Indexul epocii în înregistrarea subiectului

3.4.4 Separarea subiect-nivel pentru antrenare și validare

Un aspect critic în construirea setului de date pentru modele de clasificare EEG îl reprezintă modul în care sunt separate datele de antrenare de cele de validare. O separare naivă la nivel de epocă (graf) ar conduce la *data leakage*: epoci provenite de la același subiect ar putea apărea simultan în setul de antrenare și în cel de validare, iar modelul ar putea învăța caracteristici specifice subiectului în loc de tipare generalizabile. Acest fenomen ar supraestima artificial performanța raportată pe setul de validare [2].

Pentru a evita această problemă, separarea este realizată la nivel de subiect: toți subiecții sunt mai întâi amestecați aleatoriu (cu o sămânță fixată pentru reproductibilitate), apoi împărțiți în proporție de 80% pentru antrenare și 20% pentru validare. Toate epocile unui subiect sunt atribuite exclusiv unuia dintre cele două seturi, garantând că modelul este evaluat pe subiecți pe care nu i-a văzut în timpul antrenării.

Formal, fie $\mathcal{S} = \{s_1, s_2, \dots, s_M\}$ mulțimea celor M subiecți disponibili. Separarea produce:

$$\mathcal{S}_{train} \cup \mathcal{S}_{val} = \mathcal{S}, \quad \mathcal{S}_{train} \cap \mathcal{S}_{val} = \emptyset, \quad |\mathcal{S}_{train}| = \lfloor 0.8 \cdot M \rfloor \quad (3.11)$$

unde:

- $\mathcal{S}_{train}$ – mulțimea subiecților de antrenare;
- $\mathcal{S}_{val}$ – mulțimea subiecților de validare;
- M – numărul total de subiecți incluși în setul de date.

Seturile de grafuri corespunzătoare sunt obținute prin reuniunea grafurilor tuturor subiecților din fiecare partiție, iar acestea sunt transmise la antrenare prin obiecte `DataLoader` din PyTorch Geometric, care realizează minibatch-uri prin stivuirea eficientă a grafurilor de dimensiuni variabile [5].

3.5 Arhitecturile modelelor implementate

În cadrul acestei lucrări au fost implementate și evaluate comparativ două arhitecturi de rețele neuronale de tip graf pentru clasificarea semnalelor EEG: un model bazat pe Graph Convolutional Networks (GCN) și un model bazat pe Graph Attention Networks (GAT). Ambele arhitecturi sunt definite în directorul `src/models/` și sunt construite utilizând componentele furnizate de biblioteca PyTorch Geometric. Alegerea acestor două modele este motivată de relevanța lor în literatura de specialitate și de complementaritatea lor: GCN oferă o abordare eficientă și stabilă pentru agregarea informației din graf, în timp ce GAT permite învățarea importanței relative a vecinilor prin mecanisme de atenție.

Ambele modele rezolvă o problemă de clasificare la nivel de graf, nu la nivel de nod. Cu alte cuvinte, intrarea unui model este un graf EEG asociat unei ferestre temporale, iar ieșirea este o distribuție de probabilitate peste clasele clinice disponibile. În implementarea de față, această ieșire poate avea două dimensiuni (clasificare binară: C vs. A) sau trei dimensiuni (clasificare multi-clasă: C vs. A vs. F), în funcție de configurarea setului de date.

3.5.1 Elemente comune ale arhitecturilor

Deși GCN și GAT folosesc mecanisme diferite de propagare a informației, cele două modele implementate împărtășesc aceeași schemă generală de procesare, formată din următoarele etape:

1. **Normalizarea trăsăturilor de intrare** – trăsăturile nodurilor sunt normalizate cu un strat `BatchNorm`, pentru a stabili distribuția activărilor și a accelera convergența în timpul antrenării;
2. **Trei straturi de propagare pe graf** – fie trei straturi `GCNConv`, fie trei straturi `GATConv`, care actualizează reprezentările nodurilor pe baza structurii grafului; straturile 2 și 3 includ conexiuni reziduale pentru a facilita propagarea gradientului;
3. **Normalizare intermediară și activare neliniară** – după fiecare strat de convoluție sau atenție se aplică un nou strat `BatchNorm` și funcția de activare `ELU`;
4. **Agregare globală dublă (*dual pooling*)** – reprezentările tuturor nodurilor sunt agregate prin două operații paralele: `global_mean_pool` și `global_max_pool`; rezultatele sunt concatenate, producând un vector de dimensiune $2H$;

5. **Regularizare și clasificare** – asupra vectorului agregat se aplică **Dropout**, urmat de un **clasificator MLP cu trei straturi liniare** ($2H \rightarrow H \rightarrow H/2 \rightarrow C$), care produce logiturile finale ale claselor.

Această schemă permite transformarea unui graf de dimensiune variabilă într-o reprezentare fixă, adecvată pentru clasificare. Formal, dacă notăm cu $\mathcal{G}$ graful de intrare asociat unei ferestre temporale EEG, atunci un model de clasificare la nivel de graf poate fi privit ca o funcție:

$$f_{\theta} : \mathcal{G} \rightarrow \mathbf{z} \in \mathbb{R}^C \quad (3.12)$$

unde:

- f_{θ} – modelul GNN parametrizat prin ansamblul de ponderi antrenabile θ ;
- $\mathcal{G}$ – graful EEG de intrare;
- $\mathbf{z}$ – vectorul logiturilor de ieșire;
- C – numărul de clase ale problemei de clasificare ($C = 2$ sau $C = 3$).

Clasa prezisă este obținută prin selectarea indicelui cu logitul maxim:

$$\hat{y} = \arg \max_{c \in \{1, \dots, C\}} z_c \quad (3.13)$$

unde:

- $\hat{y}$ – eticheta prezisă de model;
- z_c – logitul asociat clasei c .

3.5.2 Modelul bazat pe Graph Convolutional Networks

Modelul GCN implementat în fișierul `src/models/gcn_model.py` poartă denumirea **EEGGCN**. Acesta este conceput pentru clasificarea grafurilor EEG și suportă atât clasificarea binară, cât și clasificarea pe trei clase, prin configurarea dimensiunii stratului final.

Arhitectura modelului este alcătuită din:

- un strat inițial **BatchNorm** aplicat trăsăturilor de intrare de dimensiune 4;
- trei straturi **GCNConv** cu activare **ELU**, normalizare **BatchNorm** și **conexiuni reziduale** la straturile 2 și 3;
- agregare globală dublă prin `global_mean_pool` și `global_max_pool`, cu concatenarea rezultatelor ($\mathbb{R}^{2H}$);
- un strat **Dropout** cu probabilitate configurabilă;
- un clasificator **MLP** cu trei straturi liniare: $2H \rightarrow H \rightarrow H/2 \rightarrow C$.

În mod implicit, dimensiunea latentă este $H = 64$, iar probabilitatea de *dropout* este 0.5.

Din punct de vedere matematic, pentru un graf cu matricea trăsăturilor nodurilor $\mathbf{X}_{nod} \in \mathbb{R}^{N \times 4}$ și matricea de adiacență ponderată $\mathbf{A}$, primul strat GCN produce:

$$\mathbf{H}^{(1)} = \text{ELU}\left(\text{BN}\left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} \mathbf{X}_{nod} W^{(0)}\right)\right) \in \mathbb{R}^{N \times H} \quad (3.14)$$

Al doilea și al treilea strat includ conexiuni reziduale:

$$\mathbf{H}^{(2)} = \text{ELU}\left(\text{BN}\left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} \mathbf{H}^{(1)} W^{(1)}\right)\right) + \mathbf{H}^{(1)} \quad (3.15)$$

$$\mathbf{H}^{(3)} = \text{ELU}\left(\text{BN}\left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} \mathbf{H}^{(2)} W^{(2)}\right)\right) + \mathbf{H}^{(2)} \quad (3.16)$$

unde $W^{(0)} \in \mathbb{R}^{4 \times H}$, $W^{(1)}, W^{(2)} \in \mathbb{R}^{H \times H}$ sunt matricele de ponderi antrenabile, $\text{BN}(\cdot)$ este operația de batch normalization, iar $\text{ELU}(\cdot)$ este funcția de activare Exponential Linear Unit.

Agregarea la nivel de graf combină media și maximul global:

$$\mathbf{h}_G = \left[\frac{1}{N} \sum_{v=1}^N \mathbf{H}_v^{(3)} \parallel \max_{v=1}^N \mathbf{H}_v^{(3)} \right] \in \mathbb{R}^{2H} \quad (3.17)$$

unde $\parallel$ denotă concatenarea, $\frac{1}{N} \sum$ este media globală, iar $\max$ este maximul element-wise peste toți N noduri.

Clasificatorul MLP produce logiturile finale printr-o rețea cu trei straturi liniare:

$$\mathbf{z} = W_3 \cdot \text{ELU}(W_2 \cdot \text{ELU}(W_1 \cdot \text{Dropout}(\mathbf{h}_G) + b_1) + b_2) + b_3 \quad (3.18)$$

unde $W_1 \in \mathbb{R}^{H \times 2H}$, $W_2 \in \mathbb{R}^{H/2 \times H}$, $W_3 \in \mathbb{R}^{C \times H/2}$ sunt ponderile clasificatorului, iar $\mathbf{z} \in \mathbb{R}^C$ este vectorul logiturilor finale.

Un aspect important al implementării este utilizarea ponderilor muchiilor. Deoarece grafurile EEG construite în această lucrare au muchii ponderate prin valori PLV, atributul `edge_attr` este convertit intern într-un vector de ponderi `edge_weight`, compatibil cu operatorul `GCNConv`. Astfel, agregarea informației între noduri este influențată nu doar de existența conexiunii, ci și de intensitatea acesteia.

3.5.3 Modelul bazat pe Graph Attention Networks

Modelul GAT implementat în fișierul `src/models/gat_model.py` poartă denumirea **EEGGAT**. Acesta are aceeași structură generală ca modelul GCN, dar înlocuiește operatorii de convoluție cu operatori de atenție `GATConv`, care permit învățarea importanței relative a vecinilor în timpul propagării.

Arhitectura modelului este alcătuită din:

- un strat inițial `BatchNorm` pentru trăsăturile de intrare;
- trei straturi `GATConv`: primul cu $K_1 = 4$ capuri de atenție și concatenare (ieșire $\mathbb{R}^{K_1 H}$), al doilea cu $K_2 = 1$ cap (ieșire $\mathbb{R}^H$), al treilea cu $K_3 = 1$ cap și conexiune reziduală via proiecție liniară din stratul 1;
- normalizare `BatchNorm` și activare **ELU** după fiecare strat;

- agregare globală dublă prin `global_mean_pool` și `global_max_pool` ($\mathbb{R}^{2H}$);
- un strat **Dropout** și un clasificator **MLP** cu trei straturi liniare.

În configurația implicită: $H = 64$, $K_1 = 4$, $K_2 = K_3 = 1$, dropout = 0.5.

Primul strat GAT produce o reprezentare latentă prin agregarea ponderată a vecinilor pentru $K_1 = 4$ capuri de atenție, cu concatenarea rezultatelor:

$$\mathbf{H}_i^{(1)} = \left\|_{k=1}^{K_1} \text{ELU} \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} W^{(k)} \mathbf{x}_j \right) \right\| \in \mathbb{R}^{K_1 H} \quad (3.19)$$

Deoarece primul strat concatenează ieșirile celor $K_1 = 4$ capuri, dimensiunea intermediară devine $K_1 \cdot H = 256$ (pentru $H = 64$). Această reprezentare este salvată pentru conexiunea reziduală.

Al doilea strat reduce dimensiunea înapoi la H ($K_2 = 1$ cap, concatenare trivială):

$$\mathbf{H}_i^{(2)} = \text{ELU} \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} W \mathbf{H}_j^{(1)} \right) \in \mathbb{R}^H \quad (3.20)$$

Al treilea strat produce reprezentarea finală a nodurilor ($K_3 = 1$ cap, fără concatenare) și adaugă conexiunea reziduală din stratul 1 printr-o proiecție liniară:

$$\mathbf{H}_i^{(3)} = \text{ELU} \left(\sum_{j \in \mathcal{N}(i)} \alpha'_{ij} W' \mathbf{H}_j^{(2)} \right) + W_{proj} \mathbf{H}_i^{(1)} \in \mathbb{R}^H \quad (3.21)$$

unde $W_{proj} \in \mathbb{R}^{H \times K_1 H}$ este matricea de proiecție antrenabilă care aliniază dimensiunile celor două reprezentări.

Agregarea la nivel de graf și clasificarea finală sunt identice cu cele utilizate în modelul GCN, folosind operația de dual pooling (Ecuația 3.17), urmată de clasificatorul MLP (Ecuația 3.18).

Un detaliu important al implementării este faptul că, deși interfața modelului acceptă parametrul `edge_attr`, acesta nu este utilizat explicit în interiorul operatorilor **GATConv**. Astfel, în varianta actuală, modelul GAT învață importanța vecinilor exclusiv pe baza trăsăturilor nodurilor și a structurii topologice a grafului, fără a integra direct valorile PLV ca ponderi în mecanismul de atenție. Această diferență față de modelul GCN constituie un aspect important în interpretarea rezultatelor experimentale.

3.5.4 Selecția dinamică a modelului

Pentru a facilita antrenarea comparativă a celor două arhitecturi, implementarea include un modul de selecție dinamică a modelului, definit în fișierul `src/models/model_factory.py`. Acesta oferă o funcție unificată de instanțiere pe baza unui identificator textual (GCN sau GAT), normalizat intern pentru a evita erorile de scriere.

Această abordare oferă două avantaje practice importante:

- simplifică integrarea modelelor în pipeline-ul de antrenare, permițând alegerea arhitecturii printr-un singur parametru;

- facilitează extinderea ulterioară a aplicației prin adăugarea altor modele GNN, fără modificarea logicii de antrenare.

Prin intermediul acestui mecanism, întregul proces de antrenare și evaluare prezentat în secțiunea următoare poate fi aplicat identic ambelor modele, diferențele de performanță observate reflectând exclusiv particularitățile arhitecturale ale operatorilor GCN și GAT.

3.5.5 Comparație între arhitecturile implementate

Tabelul 3.3 sintetizează principalele diferențe dintre cele două arhitecturi implementate.

Tabela 3.3: Comparație între modelele EEGGCN și EEGGAT implementate.

Caracteristică	EEGGCN	EEGGAT
Operator principal	GCNConv	GATConv
Număr straturi graf	3	3
Funcție de activare	ELU	ELU
Conexiuni reziduale	da (straturile 2 și 3)	da (skip strat 1 $\rightarrow$ 3, via proiecție)
Batch normalization	da	da
Global pooling	Mean $\oplus$ Max ($\mathbb{R}^{2H}$)	Mean $\oplus$ Max ($\mathbb{R}^{2H}$)
Clasificator	MLP (3 straturi liniare)	MLP (3 straturi liniare)
Utilizare explicită a <code>edge_attr</code>	da (ca <code>edge_weight</code>)	nu
Atenție multi-head	nu	da ($K_1 = 4$, $K_2 = 1$, $K_3 = 1$)
Dimensiune latentă implicită	64	64
Dropout implicit	0.5	0.5

Din punct de vedere conceptual, modelul GCN este mai simplu și mai eficient computațional, fiind adecvat pentru situațiile în care structura ponderată a grafului este deja informativă. În schimb, modelul GAT introduce o flexibilitate suplimentară prin învățarea ponderilor de atenție între vecini, ceea ce poate fi avantajos în scenarii în care importanța conexiunilor nu poate fi determinată exclusiv de valorile PLV.

În cadrul acestei lucrări, ambele modele sunt evaluate experimental în condiții comparabile, pentru a analiza în ce măsură avantajele teoretice ale mecanismului de atenție se traduc într-o performanță superioară în clasificarea semnalelor EEG asociate afecțiunilor neurodegenerative.

3.6 Procesul de antrenare

Procesul de antrenare al modelelor GNN este implementat în modulul `src/training.py` și are rolul de a învăța parametrii arhitecturilor descrise în Secțiunea 3.5 pe baza setului de grafuri construit anterior. Acest modul este responsabil de încărcarea setului de date serializat, validarea metadatelor asociate grafurilor, împărțirea dataset-ului în subseturi de antrenare și validare, inițializarea modelului selectat, optimizarea parametrilor și evaluarea performanței pe parcursul antrenării.

3.6.1 Încărcarea și validarea setului de date

Primul pas al procesului de antrenare constă în încărcarea fișierului `.pt` care conține lista de grafuri PyTorch Geometric. Fișierul este citit prin funcția `torch.load()`, iar rezultatul este validat pentru a verifica două condiții esențiale:

- fiecare graf trebuie să conțină atributul `subject_id`, necesar pentru împărțirea subiect-nivel a dataset-ului;
- fiecare graf trebuie să conțină atributul `y`, reprezentând eticheta clinică a instanței.

Această validare previne utilizarea accidentală a unor versiuni mai vechi sau incomplete ale dataset-ului, care ar putea compromite atât antrenarea, cât și evaluarea ulterioară. În plus, înainte de inițializarea modelului, este determinată automat configurația claselor active pe baza modului de clasificare selectat (`binary` sau `three_class`), ceea ce permite adaptarea dimensiunii ieșirii clasificatorului final.

3.6.2 Separarea datelor în seturi de antrenare și validare

După încărcare, dataset-ul este împărțit în subseturi de antrenare și validare prin funcția `split_dataset_by_subject()`, folosind strategia descrisă în Secțiunea 3.4.4. Această funcție grupează mai întâi toate grafurile după identificatorul subiectului, apoi amestecă lista subiectilor cu ajutorul unui generator pseudo-aleator inițializat cu o sămânță fixă.

Dacă notăm cu $\mathcal{D}$ întregul set de grafuri și cu $\mathcal{S}$ mulțimea subiectilor unici, atunci împărțirea produce două subseturi disjuncte:

$$\mathcal{D}_{train} = \bigcup_{s \in \mathcal{S}_{train}} \mathcal{D}_s, \quad \mathcal{D}_{val} = \bigcup_{s \in \mathcal{S}_{val}} \mathcal{D}_s \quad (3.22)$$

unde:

- $\mathcal{D}_s$ – mulțimea grafurilor asociate subiectului s ;
- $\mathcal{S}_{train}$ – mulțimea subiectilor alocați antrenării;
- $\mathcal{S}_{val}$ – mulțimea subiectilor alocați validării;
- $\mathcal{D}_{train}$ și $\mathcal{D}_{val}$ – subseturile finale de grafuri utilizate în cele două faze.

În implementare, proporția implicită este de 80% pentru antrenare și 20% pentru validare. Pentru a evita apariția unor subseturi vide în cazul seturilor de date mici, funcția impune și o limitare suplimentară, garantând existența a cel puțin un subiect în fiecare subset.

După împărțire, cele două subseturi sunt transmise obiectelor `DataLoader` din PyTorch Geometric, care creează minibatch-uri de grafuri de dimensiune variabilă. Pentru antrenare este activată amestecarea loturilor (`shuffle=True`), în timp ce pentru validare aceasta este dezactivată.

3.6.3 Inițializarea modelului și a componentelor de optimizare

Modelul este instanțiat prin intermediul funcției `get_model()` din modulul `src/models/model_factory.py` care primește ca parametri numele arhitecturii selectate, dimensiunea trăsăturilor nodurilor, dimensiunea latentă, numărul de clase și hiperparametrii specifici modelului GAT.

Optimizarea parametrilor se realizează cu algoritmul **Adam**, utilizând o rată de **weight decay** $\lambda = 10^{-4}$ ca regularizare L2 pentru a limita supraadaptarea. Pe parcursul antrenării este utilizat un **scheduler de rată de învățare** de tip `ReduceLROnPlateau`: dacă pierderea pe validare nu se îmbunătățește timp de 10 epoci consecutive (`patience=10`), rata de învățare este redusă cu un factor 0.5, până la valoarea minimă de 10^{-6} .

Pentru a gestiona dezechilibrul de clase prezent în setul de date (clasa AD este mai frecventă decât HC sau FTD), funcția de pierdere **CrossEntropyLoss** este configurată cu **ponderi inverse frecvenței de clasă**:

$$w_c = \frac{|\mathcal{D}_{train}|}{n_c \cdot C}, \quad c \in \{1, \dots, C\} \quad (3.23)$$

unde:

- n_c – numărul de grafuri din clasa c în setul de antrenare;
- $|\mathcal{D}_{train}|$ – numărul total de grafuri de antrenare;
- C – numărul de clase.

Funcția de pierdere ponderată este:

$$\mathcal{L} = -\frac{1}{B} \sum_{b=1}^B w_{y_b} \log \left(\frac{\exp(z_{b,y_b})}{\sum_{c=1}^C \exp(z_{b,c})} \right) \quad (3.24)$$

unde:

- B – numărul de grafuri din minibatch;
- C – numărul de clase;
- $z_{b,c}$ – logitul prezis de model pentru graful b și clasa c ;
- y_b – eticheta reală a grafului b ;
- w_{y_b} – ponderea asociată clasei reale a grafului b ;
- $\mathcal{L}$ – valoarea medie ponderată a pierderii pe minibatch.

În plus, implementarea stabilește automat dispozitivul de execuție (CPU sau CUDA), în funcție de disponibilitatea unei plăci grafice compatibile, și fixează semințele aleatoare pentru reproducibilitatea experimentelor.

3.6.4 Bucla de antrenare

Procesul de antrenare este organizat pe epoci. În fiecare epocă, modelul este trecut în modul `train()`, iar fiecare minibatch este procesat prin următoarea secvență standard:

1. transferul batch-ului pe dispozitivul de calcul;
2. resetarea gradientului optimizatorului;
3. propagarea înainte (*forward pass*) prin model;
4. calculul pierderii de clasificare;
5. propagarea înapoi a gradientului (*backpropagation*);
6. actualizarea parametrilor modelului cu optimizatorul Adam.

Pentru un batch reprezentat de grafurile $\{\mathcal{G}_1, \dots, \mathcal{G}_B\}$, modelul produce un tensor al logiturilor:

$$\mathbf{Z} \in \mathbb{R}^{B \times C} \quad (3.25)$$

unde fiecare linie corespunde unui graf din batch, iar fiecare coloană unei clase posibile.

Pe parcursul fiecărei epoci se acumulează pierderea totală ponderată cu numărul de grafuri procesate, iar la final se calculează pierderea medie:

$$\overline{\mathcal{L}}_{train} = \frac{\sum_{k=1}^K \mathcal{L}_k \cdot B_k}{\sum_{k=1}^K B_k} \quad (3.26)$$

unde:

- K – numărul de minibatch-uri din epocă;
- $\mathcal{L}_k$ – pierderea calculată pe minibatch-ul k ;
- B_k – numărul de grafuri din minibatch-ul k .

Această formulă asigură o medie corectă chiar și în situația în care ultimul batch are o dimensiune diferită de celelalte.

3.6.5 Evaluarea pe setul de validare

După fiecare epocă de antrenare, modelul este evaluat pe setul de validare prin funcția `evaluate_model()`, care trece modelul în modul `eval()` și dezactivează calculul gradientului. Pentru fiecare batch, se calculează clasa prezisă:

$$\hat{y}_b = \arg \max_{c \in \{1, \dots, C\}} z_{b,c} \quad (3.27)$$

Predicțiile și etichetele reale sunt folosite pentru a construi matricea de confuzie:

$$\mathbf{M} \in \mathbb{N}^{C \times C} \quad (3.28)$$

unde elementul $\mathbf{M}_{ij}$ reprezintă numărul de instanțe aparținând clasei reale i care au fost clasificate ca aparținând clasei j .

Pe baza acestei matrici sunt calculate următoarele metrici:

- **Acuratețea**

$$\text{Accuracy} = \frac{\sum_{c=1}^C \mathbf{M}_{cc}}{\sum_{i=1}^C \sum_{j=1}^C \mathbf{M}_{ij}} \quad (3.29)$$

- **Precizia pentru clasa c**

$$\text{Precision}_c = \frac{\mathbf{M}_{cc}}{\sum_{i=1}^C \mathbf{M}_{ic}} \quad (3.30)$$

- **Recall-ul pentru clasa c**

$$\text{Recall}_c = \frac{\mathbf{M}_{cc}}{\sum_{j=1}^C \mathbf{M}_{cj}} \quad (3.31)$$

- **Scorul F1 pentru clasa c**

$$F1_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \quad (3.32)$$

În plus, implementarea calculează versiunile *macro-average* ale metricilor de precizie, recall și F1, prin media aritmetică a valorilor pe clase. Această alegere este importantă în contextul unui posibil dezechilibru între clase, deoarece acordă aceeași pondere fiecărei clase, indiferent de numărul de exemple disponibile.

3.6.6 Salvarea checkpoint-urilor și selecția celui mai bun model

Pentru a păstra rezultatele relevante ale antrenării, implementarea include un mecanism flexibil de salvare a checkpoint-urilor. În funcție de parametrul `save_mode`, pot fi salvate:

- doar modelul cu cea mai bună acuratețe pe validare (**best**);
- doar modelul din ultima epocă (**last**);
- ambele variante (**both**).

Un checkpoint conține:

- starea parametrilor modelului (`model_state_dict`);
- numele arhitecturii utilizate;
- numărul epocii;
- metricile obținute la momentul salvării;
- configurația completă a experimentului.

Modelul considerat optim este cel care maximizează acuratețea pe setul de validare:

$$e^* = \arg \max_{e \in \{1, \dots, E\}} \text{Accuracy}_{val}^{(e)} \quad (3.33)$$

unde:

- E – numărul total de epoci de antrenare;
- $\text{Accuracy}_{val}^{(e)}$ – acuratețea pe validare la epoca e ;
- e^* – epoca asociată celui mai bun model.

La finalul antrenării, dacă s-a optat pentru salvarea celui mai bun model, parametrii acestuia sunt reîncărcați în memorie, astfel încât evaluările sau inferențele ulterioare să folosească varianta optimă.

3.6.7 Configurația experimentală salvată

Pe lângă parametrii modelului, implementarea păstrează într-un dicționar de configurare toate informațiile relevante pentru reproducerea experimentului:

- arhitectura modelului;
- modul de clasificare;
- numărul de clase și etichetele asociate;
- hiperparametrii de antrenare (număr de epoci, rată de învățare, batch size, dropout);
- hiperparametrii specifici GAT (numărul de capuri de atenție);
- pragul PLV și durata epocilor;
- numărul total de grafuri și de subiecți;
- împărțirea subiecților între antrenare și validare;
- dispozitivul de execuție utilizat.

Această practică facilitează nu doar reproducerea rezultatelor, ci și analiza comparativă între experimente, fiind esențială în contextul evaluării mai multor configurații de model și hiperparametri.

3.7 Mecanismul de inferență

După antrenarea modelului, etapa de inferență are rolul de a genera predicții pentru înregistrări EEG noi, care nu au fost utilizate în procesul de învățare. În cadrul implementării, această funcționalitate este definită în modulul `src/inference.py` și permite atât evaluarea pe date de tip holdout, cât și integrarea predicției în interfața grafică dezvoltată cu Streamlit.

Inferența urmează aceeași logică de pre-procesare și construire a grafurilor ca în cazul datelor de antrenare, asigurând astfel consistența între faza de învățare și cea de utilizare efectivă a modelului.

3.7.1 Încărcarea checkpoint-ului și reconstrucția modelului

Primul pas al inferenței constă în încărcarea checkpoint-ului salvat în etapa de antrenare. Fișierul de checkpoint conține nu doar parametrii modelului, ci și configurația completă a experimentului, ceea ce permite reconstrucția fidelă a arhitecturii utilizate.

Pe baza câmpurilor `model_name`, `num_node_features`, `hidden_channels`, `num_classes`, `dropout` și, după caz, a parametrilor specifici GAT, este reinstanciat modelul corespunzător, iar apoi sunt încărcate ponderile salvate în `model_state_dict`. Modelul este trecut în modul `eval()`, pentru a dezactiva comportamentele specifice antrenării, precum *dropout*-ul stochastic.

Astfel, dacă notăm checkpoint-ul cu $\mathcal{C}$ și modelul reconstruit cu f_{θ^*} , inferența utilizează exact aceiași parametri θ^* care au fost selectați în timpul antrenării drept optimi pe setul de validare.

3.7.2 Pre-procesarea unei înregistrări noi

O înregistrare EEG nouă este procesată prin același pipeline descris în Secțiunea 3.2:

1. încărcarea fișierului `.set`;
2. selecția celor 19 canale standard;
3. aplicarea filtrării trece-bandă;
4. aplicarea opțională a curățării prin ICA;
5. segmentarea în ferestre temporale de 5 secunde fără suprapunere.

Rezultatul este tensorul:

$$\mathbf{X}^{new} \in \mathbb{R}^{N_e \times 19 \times N_t} \quad (3.34)$$

unde:

- N_e – numărul de epoci extrase din noua înregistrare;
- 19 – numărul de canale EEG selectate;
- N_t – numărul de eșantioane temporale din fiecare epocă.

Fiecare fereastră temporală este apoi convertită într-un graf PyG prin aceeași procedură utilizată la construirea dataset-ului de antrenare: extragerea trăsăturilor nodurilor, calculul matricei PLV, aplicarea pragului și conversia în format `Data`. Pragul PLV și durata în secunde a ferestrelor temporale sunt recuperate direct din configurația checkpoint-ului, garantând coerența cu experimentul de antrenare.

3.7.3 Predicția la nivel de fereastră temporală

Fiecare graf asociat unei ferestre temporale este transmis individual prin modelul antrenat, rezultând un vector de logituri:

$$\mathbf{z}^{(e)} \in \mathbb{R}^C, \quad e = 1, \dots, N_e \quad (3.35)$$

unde:

- $\mathbf{z}^{(e)}$ – vectorul logiturilor asociat ferestrei temporale e ;
- C – numărul de clase ale problemei.

Pentru fiecare fereastră temporală se determină clasa prezisă:

$$\hat{y}^{(e)} = \arg \max_{c \in \{1, \dots, C\}} z_c^{(e)} \quad (3.36)$$

În plus, pentru a obține un scor de încredere, logiturile sunt transformate în probabilități prin funcția softmax:

$$p_c^{(e)} = \frac{\exp(z_c^{(e)})}{\sum_{k=1}^C \exp(z_k^{(e)})} \quad (3.37)$$

unde:

- $p_c^{(e)}$ – probabilitatea estimată pentru clasa c la fereastra temporală e ;
- $\sum_{c=1}^C p_c^{(e)} = 1$ pentru fiecare fereastră temporală.

Astfel, inferența produce nu doar o clasă discretă pentru fiecare fereastră temporală, ci și o distribuție de probabilitate care poate fi utilizată pentru interpretare și vizualizare în interfața grafică.

3.7.4 Agregarea predicțiilor la nivel de subiect

Deoarece decizia clinică trebuie formulată la nivel de înregistrare sau subiect, nu la nivel de fereastră temporală individuală, implementarea agregă predicțiile per fereastră temporală într-o singură predicție finală. Strategia utilizată este **agregarea prin probabilități medii**: pentru fiecare clasă c , se calculează probabilitatea medie peste toate ferestrele temporale ale înregistrării, iar clasa cu probabilitatea medie maximă este selectată ca predicție finală.

Formal, fie $\{p_c^{(e)}\}_{e=1}^{N_e}$ distribuțiile de probabilitate obținute pentru fiecare fereastră temporală e . Predicția finală este:

$$\hat{y}_{final} = \arg \max_{c \in \{1, \dots, C\}} \frac{1}{N_e} \sum_{e=1}^{N_e} p_c^{(e)} \quad (3.38)$$

unde:

- $p_c^{(e)}$ – probabilitatea estimată pentru clasa c la fereastra temporală e , obținută prin softmax;
- N_e – numărul total de ferestre temporale ale înregistrării;
- $\hat{y}_{final}$ – eticheta finală asociată înregistrării.

Această abordare este mai robustă decât votul majoritar simplu, deoarece ține cont de gradul de certitudine al modelului pentru fiecare fereastră temporală, nu doar de

clasa cu probabilitate maximă per fereastră. Confidența predicției finale este raportată ca probabilitatea medie a clasei câștigătoare:

$$\text{conf} = \max_c \frac{1}{N_e} \sum_{e=1}^{N_e} p_c^{(e)} \quad (3.39)$$

Pe lângă predicția finală, sistemul calculează și distribuția probabilităților medii pe toate clasele, ceea ce permite exprimarea rezultatului într-o formă mai nuanțată și interpretabilă. Această abordare oferă un grad suplimentar de transparență și poate evidenția cazurile în care predicția este instabilă sau ambiguă.

3.7.5 Maparea etichetelor numerice la denumiri clinice

Pentru a face rezultatul inferenței ușor de interpretat de către utilizator, etichetele numerice produse de model sunt transformate în denumiri clinice prin intermediul dicționarului `class_labels`, salvat în checkpoint. Astfel:

- în clasificarea binară: $0 \rightarrow \text{Healthy Control}$, $1 \rightarrow \text{Alzheimer's Disease}$;
- în clasificarea pe trei clase: $0 \rightarrow \text{Healthy Control}$, $1 \rightarrow \text{Alzheimer's Disease}$, $2 \rightarrow \text{Frontotemporal Dementia}$.

Această mapare este utilizată atât în scriptul de inferență, cât și în interfața grafică, astfel încât utilizatorul să nu interacționeze cu etichete numerice abstracte, ci direct cu diagnosticele corespunzătoare.

3.7.6 Rolul inferenței în aplicația finală

Modulul de inferență reprezintă componenta care conectează modelul antrenat cu utilizarea practică a sistemului. Din perspectiva aplicației finale, această etapă permite:

- clasificarea unor înregistrări EEG noi, neutilizate în antrenare;
- evaluarea comportamentului modelului pe seturi de date de tip holdout;
- integrarea rezultatelor în interfața Streamlit pentru vizualizare și interpretare.

Prin păstrarea consistenței între pașii utilizați la antrenare și cei utilizați la inferență, sistemul reduce riscul apariției unor discrepanțe între distribuțiile datelor din cele două faze și asigură o utilizare robustă a modelului în scenarii reale. În plus, faptul că decizia finală este obținută prin agregarea predicțiilor pe mai multe epoci contribuie la stabilitatea clasificării și reduce influența eventualelor fluctuații locale sau artefacte reziduale prezente într-o singură fereastră temporală.

Capitolul 4

Interfața grafică

Prezentul capitol descrie interfața grafică interactivă dezvoltată cu Streamlit, care permite vizualizarea semnalelor EEG, explorarea grafurilor de conectivitate funcțională, configurarea și lansarea antrenării modelelor GNN, precum și realizarea de predicții pe înregistrări noi. Interfața integrează toate componentele pipeline-ului implementat și oferă utilizatorului un mod accesibil și interactiv de utilizare a sistemului.

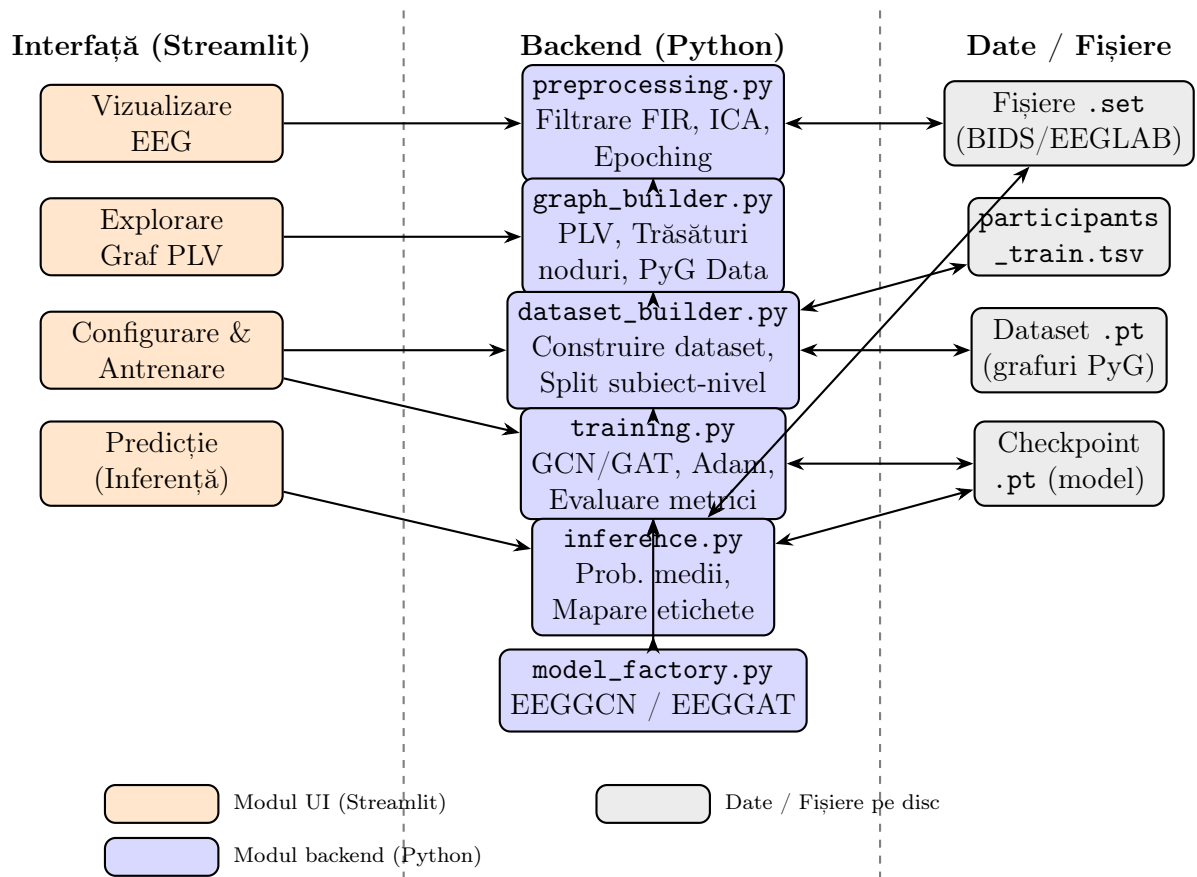


Figura 4.1: Diagrama de arhitectură a aplicației grafice interactive dezvoltate cu Streamlit, evidențiind modulele principale și fluxul de date între ele.

4.1 Prezentare generală

Pentru a facilita utilizarea practică a sistemului propus, a fost dezvoltată o interfață grafică interactivă bazată pe framework-ul Streamlit. Alegerea acestei tehnologii este motivată de simplitatea integrării cu ecosistemul Python, de suportul nativ pentru elemente interactive și de capacitatea de a construi rapid aplicații destinate explorării datelor și modelelor de învățare automată.

Interfața are rolul de a conecta utilizatorul cu toate etapele importante ale pipeline-ului implementat: încărcarea și inspectarea semnalelor EEG, vizualizarea conectivității funcționale, configurarea și antrenarea modelelor GNN și realizarea de predicții pe înregistrări noi. Astfel, aplicația nu reprezintă doar o componentă auxiliară, ci un strat de interacțiune care face posibilă utilizarea sistemului într-un mod intuitiv și reproductibil.

Din punct de vedere al organizării interne, aplicația este structurată în două module principale:

- fișierul `app/main.py`, care definește logica principală a aplicației, navigarea între secțiuni și integrarea funcțiilor de procesare, antrenare și inferență;
- fișierul `app/ui_components.py`, care conține componente reutilizabile pentru interfață, elemente grafice și funcții auxiliare pentru afișarea rezultatelor.

Aplicația comunică direct cu modulele backend dezvoltate în capitolele anterioare:

- `src/preprocessing.py` – pentru încărcarea și pre-procesarea semnalelor EEG;
- `src/graph_builder.py` – pentru construirea și vizualizarea grafurilor de conectivitate funcțională;
- `src/dataset_builder.py` – pentru generarea setului de date;
- `src/training.py` – pentru antrenarea și evaluarea modelelor;
- `src/inference.py` – pentru realizarea predicțiilor.

Din perspectiva utilizatorului, interfața este organizată în patru zone funcționale principale:

1. vizualizarea semnalelor EEG;
2. explorarea conectivității funcționale și a grafurilor asociate;
3. configurarea și antrenarea modelelor;
4. realizarea predicțiilor pe date noi.

Această împărțire reflectă etapele naturale ale fluxului de lucru și permite utilizatorului să treacă gradual de la inspecția datelor brute la interpretarea rezultatului final al clasificării.

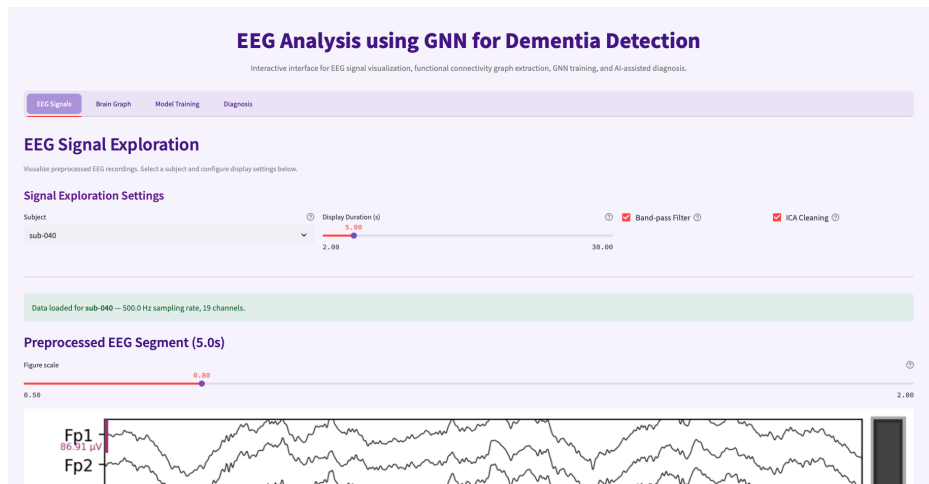


Figura 4.2: Pagina principală a aplicației grafice dezvoltate cu Streamlit.

4.2 Vizualizarea semnalelor EEG

Una dintre funcționalitățile esențiale ale interfeței este posibilitatea de a inspecta vizual semnalele EEG încărcate. Această componentă este utilă atât pentru verificarea corectitudinii datelor introduse, cât și pentru o înțelegere mai bună a caracteristicilor semnalului înainte de aplicarea etapelor de procesare și clasificare.

Utilizatorul poate selecta un fișier EEG în format `.set`, iar aplicația îl încarcă folosind aceleași funcții descrise în Secțiunea 3.2. După încărcare, sunt afișate informații descriptive de bază, precum:

- numărul de canale disponibile;
- lista canalelor selectate;
- rata de eșantionare;
- durata totală a înregistrării.

Pe lângă aceste informații, interfața permite reprezentarea grafică a semnalelor EEG pe canale, pe o durată de timp selectabilă de către utilizator. Această vizualizare este importantă pentru identificarea unor caracteristici evidente ale semnalului, precum fluctuațiile de amplitudine, prezența zgomotului sau eventuale artefacte reziduale.

În plus, aplicația oferă utilizatorului posibilitatea de a vizualiza matricea de conectivitate funcțională calculată prin PLV. Aceasta este afișată sub forma unei hărți de căldură (*heatmap*), în care intensitatea culorii reflectă nivelul de sincronizare dintre perechile de electrozi. O astfel de reprezentare permite observarea rapidă a relațiilor funcționale dominante dintre regiunile cerebrale.

Pornind de la această matrice, aplicația poate genera și o reprezentare explicită a grafului EEG, în care:

- nodurile corespund electrozilor;
- muchiile reprezintă conexiunile funcționale care depășesc pragul selectat;
- grosimea muchiilor este proporțională cu intensitatea PLV.

Această componentă are un rol important în interpretarea reprezentării pe graf utilizate de modelele GNN, oferind o legătură directă între semnalul inițial și structura topologică folosită în clasificare.

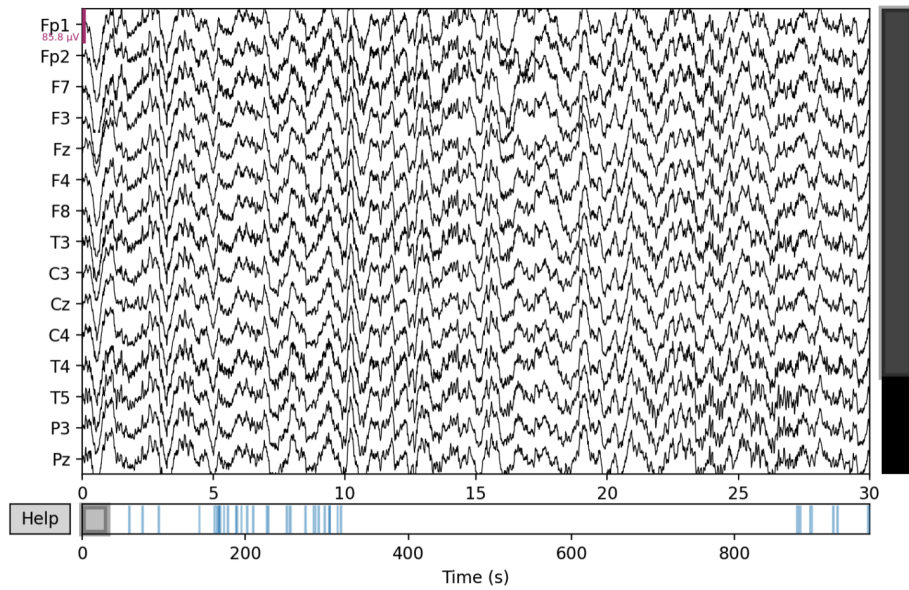


Figura 4.3: Modulul de vizualizare a semnalelor EEG în interfața grafică.

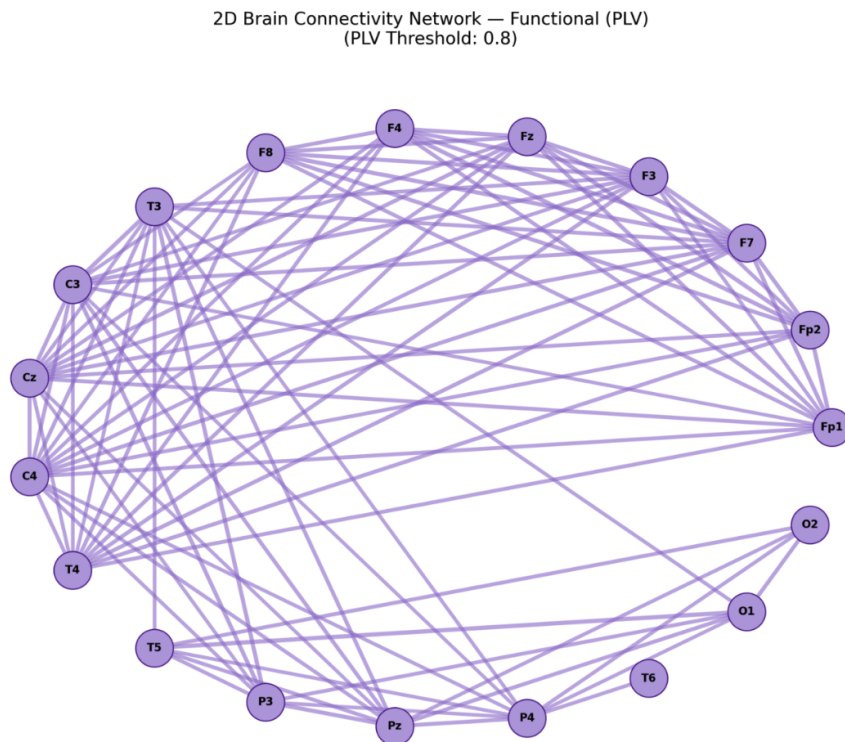


Figura 4.4: Vizualizarea conectivității funcționale și a grafului EEG asociat unei înregistrări.

4.3 Configurarea și antrenarea modelelor

Interfața include un modul dedicat configurării experimentelor și lansării procesului de antrenare. Scopul acestuia este de a permite utilizatorului să controleze parametrii principali ai modelului și ai pipeline-ului, fără a fi necesară modificarea directă a codului sursă.

Din interfață pot fi configurați, printre alții, următorii parametri:

- arhitectura modelului (GCN sau GAT);
- modul de clasificare (*binary* sau *three_class*);
- numărul de epoci de antrenare;
- rata de învățare;
- dimensiunea batch-ului;
- dimensiunea spațiului latent;
- probabilitatea de *dropout*;
- pragul PLV utilizat la construirea grafurilor;
- durata ferestrei temporale folosite în segmentare.

În cazul modelului GAT, utilizatorul poate specifica suplimentar numărul de capuri de atenție utilizate în fiecare strat. Această flexibilitate permite explorarea mai multor configurații experimentale și compararea comportamentului celor două arhitecturi implementate.

După configurare, utilizatorul poate lansa procesul de antrenare direct din interfață. În timpul execuției, aplicația afișează în timp real informații despre progresul antrenării, precum:

- epoca de antrenare curentă;
- pierderea medie pe antrenare;
- acuratețea pe validare;
- valorile macro pentru precizie, recall și scor F1;
- cea mai bună performanță obținută până în acel moment.

Această monitorizare în timp real este utilă pentru identificarea comportamentului modelului pe parcursul antrenării și pentru detectarea unor posibile probleme, precum stagnarea performanței sau supraînvățarea.

La finalul procesului, aplicația afișează un rezumat al experimentului, incluzând cea mai bună epocă de antrenare și valorile finale ale metricilor. În plus, checkpoint-urile salvate pot fi utilizate ulterior atât pentru analiză, cât și pentru inferență.

Step 2: GNN Training Configuration

Configure the model architecture and training hyperparameters, then start training.

Model Architecture GCN	Hidden Channels 48	Dropout Rate 0.50
Training Epochs 100	Learning Rate 0.001	Batch Size 32
Train Split Ratio 0.80	Checkpoint Save Mode best	Checkpoint Name GCN_test

Advanced Training Settings

Start Training

Figura 4.5: Panoul de configurare a hiperparametrilor și de lansare a antrenării.

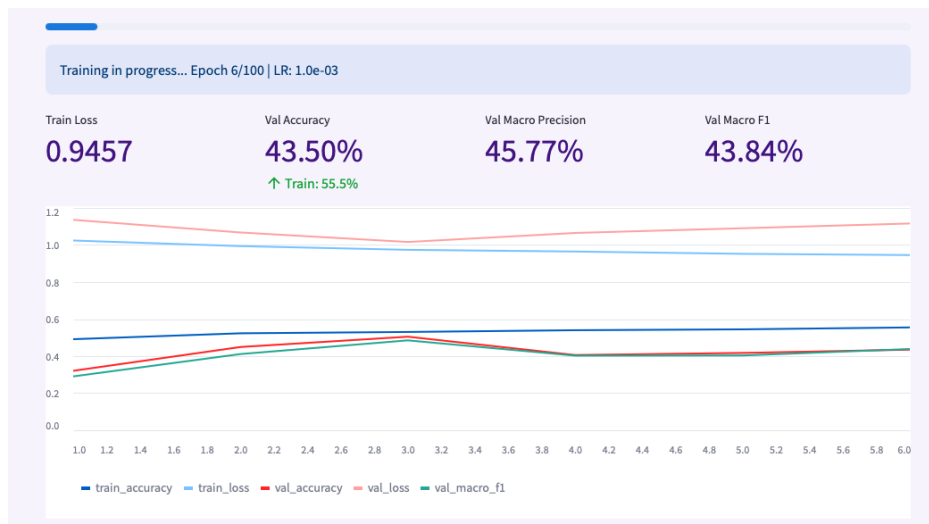


Figura 4.6: Afișarea progresului antrenării și a metricilor calculate pe parcursul procesului de optimizare.

4.4 Realizarea predicțiilor

Un alt modul important al aplicației este cel dedicat inferenței pe înregistrări EEG noi. Acesta permite utilizatorului să încarce un fișier necunoscut modelului și să obțină o predicție automată privind clasa clinică asociată.

Fluxul de lucru urmat în această secțiune reproduce pașii descriși în Secțiunea 3.7:

1. selectarea unui checkpoint antrenat anterior;
2. încărcarea unei înregistrări EEG noi;
3. aplicarea pre-procesării standard;
4. segmentarea în ferestre temporale;

5. construirea grafurilor corespunzătoare;
6. clasificarea fiecărei ferestre temporale;
7. agregarea predicțiilor prin probabilități medii.

Rezultatul final este afișat sub forma unei etichete clinice ușor de interpretat, împreună cu distribuția predicțiilor pe ferestre temporale. De exemplu, utilizatorul poate vedea nu doar clasa prezisă final, ci și câte segmente temporale au fost clasificate în fiecare categorie.

Această prezentare oferă un nivel suplimentar de transparență și permite identificarea situațiilor în care predicția este stabilă sau, dimpotrivă, ambiguă. În plus, interfața poate afișa și probabilitățile asociate claselor, oferind o imagine mai nuanțată asupra încrederii modelului în rezultatul furnizat.

Prin această componentă, aplicația depășește rolul unui simplu instrument experimental și se apropie de un prototip funcțional destinat analizei asistate a semnalelor EEG. Integrarea inferenței în interfața grafică contribuie semnificativ la accesibilitatea sistemului și la potențialul său de utilizare în contexte educaționale sau de cercetare aplicată.

GNN Model Inference / Diagnosis

Test the trained model on reserved holdout EEG recordings or upload a new EEG file for AI-assisted diagnosis.

Diagnosis Settings

Run the trained GNN model on an EEG recording to predict the diagnosis. You can test on holdout subjects (reserved data) or upload a new EEG file.

EEG Source ⓘ
☒ Project Holdout Sample ☐ Manual Upload (.set file)

Select Holdout Subject ⓘ
sub-024

Model Checkpoint ⓘ Time Window Duration (s) ⓘ PLV Threshold ⓘ
GCN_test_best.pt 2.00 5.00 10.00 0.10 0.90 0.99

☒ Band-pass Filter ⓘ ☐ ICA Cleaning ⓘ Batch Size ⓘ ☒ Show Detailed Table ⓘ
16

Connectivity Type ⓘ
Auto (use the type the model was trained with)

Run Diagnosis

Figura 4.7: Panoul de configurare pentru realizarea predicțiilor.

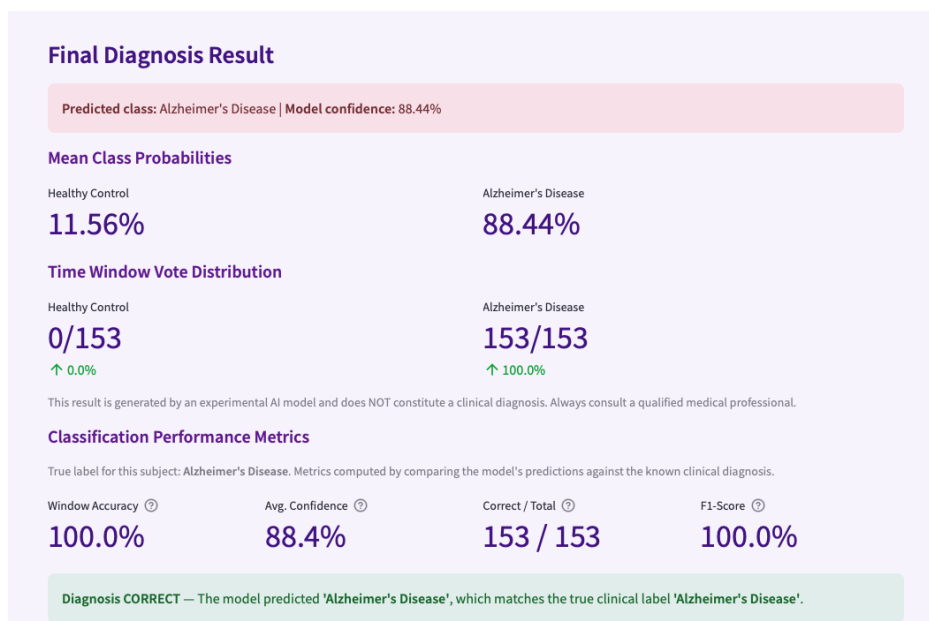


Figura 4.8: Panoul de afişare a rezultatelor pentru realizarea predicțiilor.



Figura 4.9: Panoul de vizualizare a rezultatelor pentru realizarea predicțiilor.

Per-Window Predictions								
Ground truth: Alzheimer's Disease Correct windows: 153/153 The 'Correct' column indicates whether each window's prediction matches the true diagnosis. 'P(True Class)' shows the probability assigned to the true class by the model.								
Window #	Start (s)	End (s)	Prediction	Confidence	Correct	P(True Class)	P(Healthy Control)	P(Alzheimer's Disease)
0	0	5	Alzheimer's Disease	0.8904	Yes	0.8904	0.1096	0.8904
1	5	10	Alzheimer's Disease	0.9003	Yes	0.9003	0.0997	0.9003
2	10	15	Alzheimer's Disease	0.8199	Yes	0.8199	0.1801	0.8199
3	15	20	Alzheimer's Disease	0.8963	Yes	0.8963	0.1037	0.8963
4	20	25	Alzheimer's Disease	0.7219	Yes	0.7219	0.2781	0.7219
5	25	30	Alzheimer's Disease	0.9682	Yes	0.9682	0.0318	0.9682
6	30	35	Alzheimer's Disease	0.8671	Yes	0.8671	0.1329	0.8671
7	35	40	Alzheimer's Disease	0.9523	Yes	0.9523	0.0477	0.9523
8	40	45	Alzheimer's Disease	0.6982	Yes	0.6982	0.3018	0.6982
9	45	50	Alzheimer's Disease	0.8935	Yes	0.8935	0.1065	0.8935

Figura 4.10: Tabelul de afișare a rezultatelor pentru realizarea predicțiilor per fereastră temporală.

4.5 Observații privind utilitatea interfeței

Interfața grafică dezvoltată în cadrul acestei lucrări are un rol important nu doar din perspectiva utilizării practice, ci și din perspectiva validării și interpretării întregului sistem propus. Prin centralizarea etapelor de încărcare, pre-procesare, construire a grafurilor, antrenare și inferență într-un singur mediu interactiv, aplicația facilitează explorarea comportamentului modelelor și verificarea coerenței rezultatelor obținute.

De asemenea, această interfață contribuie la creșterea accesibilității soluției dezvoltate. Utilizatorii care nu sunt familiarizați cu detaliile implementării la nivel de cod pot interacționa cu sistemul într-un mod intuitiv, configurând experimente și analizând rezultate prin elemente vizuale și controale simple. În același timp, integrarea directă cu modulele backend garantează faptul că interfața utilizează exact aceleași proceduri de procesare și clasificare descrise în capitolele anterioare.

În concluzie, interfața grafică reprezintă o extensie firească a pipeline-ului propus, transformând o implementare experimentală într-un sistem mai ușor de utilizat, de analizat și de demonstrat în contexte academice sau aplicative.

Capitolul 5

Rezultate experimentale

Prezentul capitol prezintă evaluarea cantitativă a modelelor implementate. Sunt raportate metricele de validare la nivel de fereastră temporală, rezultatele inferenței pe subiecți din setul holdout și o analiză comparativă care evidențiază impactul tipului de conectivitate și al arhitecturii asupra performanței.

5.1 Configurația experimentală

Experimentele au fost realizate pe setul de date descris în Secțiunea 3.1, utilizând înregistrări EEG resting-state (protocol *eyes-closed*), 19 canale din sistemul 10-20, stocate în format BIDS. Împărțirea datelor s-a realizat la nivel de subiect (80% antrenare / 20% validare), eliminând riscul de *data leakage*.

Au fost evaluate patru combinații de model și tip de conectivitate, pe două scenarii de clasificare:

- **Conectivitate funcțională (PLV) vs. Conectivitate topologică (proximitate spațială 10-20);**
- **GCN vs. GAT;**
- **Clasificare binară (Healthy Control vs. Alzheimer’s Disease) vs. Clasificare pe trei clase (HC vs. AD vs. FTD).**

Dimensiunile seturilor de date construite sunt sintetizate în Tabelul 5.1.

Tabela 5.1: Dimensiunea seturilor de date utilizate în experimente.

Mod	Nr. grafuri total	Subiecți antrenare	Subiecți validare
Binar (C vs. A)	22 888	50	13
3 clase (C/A/F)	30 022	68	17

Hiperparametrii comuni tuturor experimentelor sunt: dimensiune latentă $H = 64$, dropout = 0.5, optimizator Adam cu weight decay 10^{-4} , scheduler `ReduceLROnPlateau` (patience = 10, factor = 0.5), funcție de pierdere `CrossEntropyLoss` cu ponderi inverse frecvenței de clasă.

5.2 Rezultate la nivel de fereastră temporală

Metricile raportate sunt macro-averaged (medie egală pe clase), calculate pe setul de validare al fiecărui model. Valoarea reținută corespunde epocii cu acuratețea maximă pe validare (*best checkpoint*).

5.2.1 Clasificare binară (Healthy Control vs. Alzheimer’s Disease)

Tabela 5.2: Rezultate de validare pentru clasificarea binară (C vs. A).

Conectivitate	Model	Accuracy	Precision	Recall	F1	Epocă
Funcțională	GCN	71,0%	0,751	0,746	0,710	7
Funcțională	GAT	73,8%	0,741	0,751	0,736	25
Topologică	GCN	66,9%	0,721	0,709	0,668	7
Topologică	GAT	71,1%	0,728	0,734	0,710	38

Cel mai bun model în scenariul binar este **GAT cu conectivitate funcțională**, cu o acuratețe de 73,8% și F1 macro de 0,736. Se observă că GAT depășește GCN în ambele tipuri de conectivitate, iar conectivitatea funcțională (PLV) conduce la performanțe superioare față de cea topologică pentru ambele arhitecturi.

5.2.2 Clasificare pe trei clase (HC vs. AD vs. FTD)

Tabela 5.3: Rezultate de validare pentru clasificarea pe trei clase (C/A/F).

Conectivitate	Model	Accuracy	Precision	Recall	F1	Epocă
Funcțională	GCN	56,0%	0,579	0,627	0,510	5
Funcțională	GAT	60,8%	0,611	0,639	0,564	22
Topologică	GCN	52,1%	0,435	0,556	0,418	1
Topologică	GAT	60,5%	0,546	0,630	0,555	49

Scenariul cu trei clase este semnificativ mai dificil, acuratețea maximă atingând 60,8% (GAT funcțional). Scorul F1 macro scăzut al modelului GCN topologic (0,418) indică o convergență rapidă spre o soluție suboptimală, confirmată de faptul că *best checkpoint* a fost salvat la prima epocă.

5.3 Rezultate la nivel de subiect (inferență pe holdout)

Pentru evaluarea capacității de generalizare la nivel clinic, au fost testate trei subiecți din setul holdout, cu diagnostice reale cunoscute: **sub-024** (Alzheimer’s Disease), **sub-055** (Healthy Control) și **sub-070** (Frontotemporal Dementia). Predicția finală pentru fiecare subiect a fost obținută prin agregarea probabilităților medii pe toate ferestrele temporale ale înregistrării, conform Ecuației (3.38).

Tabela 5.4: Rezultatele inferenței pe setul holdout. C = Healthy Control, A = Alzheimer’s Disease, F = Frontotemporal Dementia.

Subject	Diagnostic real	Conectivitate	Model	Mod	Predicție	Confidență
sub-024	AD	Funcțională	GCN	Binar	AD	86,2%
sub-024	AD	Funcțională	GAT	Binar	AD	88,4%
sub-024	AD	Topologică	GCN	Binar	AD	74,8%
sub-024	AD	Topologică	GAT	Binar	AD	87,3%
sub-024	AD	Funcțională	GCN	3 clase	AD	55,8%
sub-024	AD	Funcțională	GAT	3 clase	AD	57,1%
sub-024	AD	Topologică	GCN	3 clase	AD	51,3%
sub-024	AD	Topologică	GAT	3 clase	AD	64,9%
sub-055	HC	Funcțională	GCN	Binar	HC	80,0%
sub-055	HC	Funcțională	GAT	Binar	HC	69,5%
sub-055	HC	Topologică	GCN	Binar	HC	77,9%
sub-055	HC	Topologică	GAT	Binar	HC	82,2%
sub-055	HC	Funcțională	GCN	3 clase	HC	60,5%
sub-055	HC	Funcțională	GAT	3 clase	HC	53,1%
sub-055	HC	Topologică	GCN	3 clase	HC	48,4%
sub-055	HC	Topologică	GAT	3 clase	HC	48,3%
sub-070	FTD	Funcțională	GCN	3 clase	HC	34,5%
sub-070	FTD	Funcțională	GAT	3 clase	AD	35,0%
sub-070	FTD	Topologică	GCN	3 clase	AD	37,6%
sub-070	FTD	Topologică	GAT	3 clase	AD	41,1%

Sumarul acurateței la nivel de subiect este prezentat în Tabelul 5.5.

Tabela 5.5: Sumarul acurateței la nivel de subiect pe setul holdout.

Categorie	Corecte	Total	Acuratețe
Modele binare (sub-024, sub-055)	8	8	100%
Modele 3-clase (sub-024, sub-055, sub-070)	8	12	66,7%
Total	16	20	80%

Toate cele opt rulări în modul binar au clasificat corect ambii subiecți testați, cu confidențe cuprinse între 69,5% și 88,4%, ceea ce indică o separare robustă între clasa Alzheimer și cea sănătoasă. În modul cu trei clase, subiecții sub-024 (AD) și sub-055 (HC) au fost clasificați corect de toate modelele, în timp ce sub-070 (FTD) nu a fost clasificat corect de niciun model, cu confidențe la nivel de șansă (34–41%), indicând ambiguitate ridicată în predicție.

5.4 Analiză comparativă și discuții

Conectivitate funcțională vs. topologică. În toate configurațiile testate, conectivitatea funcțională bazată pe PLV a condus la performanțe egale sau superioare față de conectivitatea topologică. Diferența este mai pronunțată în cazul GCN (71,0% vs. 66,9% la binar; 56,0% vs. 52,1% la 3 clase), sugerând că sincronizarea de fază în

banda Alpha captează tipare discriminative pe care simpla proximitate anatomică a electrozilor nu le poate oferi.

GAT vs. GCN. Modelul GAT depășește GCN în toate cele patru combinații de conectivitate și mod de clasificare. Avantajul este mai consistent în regimul cu trei clase, unde mecanismul de atenție adaptivă permite modelului să se concentreze pe conexiunile relevante între regiuni cerebrale, reducând impactul zgomotului structural.

Binar vs. trei clase. Trecerea de la clasificarea binară la cea cu trei clase produce o scădere semnificativă a acurateței (de la $\sim 73\%$ la $\sim 61\%$ pentru cel mai bun model). Scăderea este explicată parțial de dimensiunea mai mică a clasei FTD în setul de antrenare și de suprapunerea parțială a semnăturii EEG a FTD cu cea a AD, ambele afecțiuni prezentând modificări similare în banda Theta și Delta.

Cazul sub-070 (FTD). Eșecul tuturor modelelor în clasificarea subiectului FTD, cu confidențe scăzute (34–41%), nu este neapărat surprinzător în contextul setului de date disponibil. Clasa FTD este sub-reprezentată numeric față de AD și HC, iar trăsăturile spectrale utilizate (puterea pe patru benzi) pot fi insuficiente pentru a distinge FTD de AD la nivel individual. Aceste observații indică direcții clare de îmbunătățire, discutate în Capitolul 6.

Limite ale evaluării. Setul holdout conține doar trei subiecți, ceea ce face ca rezultatele la nivel de subiect să fie ilustrative, nu statistic concludente. Baza cantitativă solidă a evaluării rămâne metricile macro-averaged pe setul de validare (Tabelele 5.2 și 5.3), calculate pe 13, respectiv 17 subiecți nevăzuți în antrenare.

Capitolul 6

Concluzii și direcții viitoare de dezvoltare

Prezentul capitol sintetizează contribuțiile principale ale lucrării, discută limitările identificate pe parcursul procesului de implementare și evaluare și propune direcții de cercetare și dezvoltare viitoare.

6.1 Concluzii

Această lucrare a investigat utilizarea rețelelor neuronale de tip graf pentru analiza și clasificarea semnalelor EEG asociate afecțiunilor neurodegenerative, cu accent pe boala Alzheimer și demența frontotemporală. Pornind de la natura non-euclidiană a datelor EEG, a fost propus și implementat un pipeline complet care transformă înregistrările brute în grafuri procesabile de modele GNN, prin extragerea puterii spectrale pe benzi de frecvență ca trăsături nodale și utilizarea Phase Locking Value în banda Alpha ca măsură de conectivitate funcțională pentru definirea muchiilor.

Au fost implementate și evaluate comparativ două arhitecturi: **EEGGCN** (Graph Convolutional Network cu trei straturi, conexiuni reziduale și dual pooling) și **EEGGAT** (Graph Attention Network cu mecanism multi-head și conexiune reziduală cu proiecție liniară). Evaluarea a acoperit patru axe de comparație: tipul de conectivitate (funcțională vs. topologică), arhitectura modelului (GCN vs. GAT), modul de clasificare (binar vs. trei clase) și generalizarea la subiecți nevăzuți (setul holdout).

Principalele concluzii susținute de rezultatele experimentale sunt:

1. **Conectivitatea funcțională (PLV) este superioară celei topologice.** În toate configurațiile testate, grafurile construite pe baza sincronizării de fază în banda Alpha au condus la performanțe mai bune față de grafurile bazate pe proximitatea anatomică a electrozilor. Aceasta confirmă că modificările de conectivitate funcțională inter-regională constituie un semnal discriminativ relevant pentru detecția afecțiunilor neurodegenerative.
2. **GAT depășește GCN în mod consistent.** Mecanismul de atenție adaptivă permite modelului GAT să diferențieze importanța conexiunilor în funcție de context, ceea ce se traduce într-un avantaj de 2–5 puncte procentuale față de GCN, mai pronunțat în scenariul cu trei clase.

3. **Clasificarea binară HC vs. AD este robustă.** Cel mai bun model (GAT funcțional) atinge o acuratețe de 73,8% și F1 macro de 0,736 la nivel de fereastră temporală, cu o acuratețe de 100% la nivel de subiect pe setul holdout, cu confidențe de 69–88%. Aceste rezultate demonstrează că semnalele EEG resting-state, prelucrate prin pipeline-ul propus, conțin informație suficientă pentru a distinge subiecții sănătoși de cei cu boala Alzheimer.
4. **Clasificarea cu trei clase rămâne o provocare.** Acuratețea maximă de 60,8% și eșecul tuturor modelelor în clasificarea subiectului FTD (cu confidențe de 34–41%) indică limitele abordării actuale în fața dezechilibrului de clase și a suprapunerii semnăturilor spectrale dintre AD și FTD.
5. **Interfața grafică interactivă** dezvoltată cu Streamlit transformă sistemul dintr-un experiment de cercetare într-un prototip funcțional, accesibil utilizatorilor fără expertiză în programare, și demonstrează potențialul aplicativ al soluției propuse.

În ansamblu, lucrarea demonstrează că rețelele neuronale de tip graf, combinate cu o reprezentare adecvată a datelor EEG bazată pe conectivitate funcțională, reprezintă o alternativă promițătoare față de metodele tradiționale de clasificare a semnalelor neurofiziologice.

6.2 Limitări identificate

Pe parcursul implementării și evaluării au fost identificate mai multe limitări care trebuie luate în considerare la interpretarea rezultatelor:

- **Dimensiunea setului de date.** Setul utilizat conține 85 de subiecți în total (63 pentru antrenare în modul binar), o dimensiune relativ mică pentru antrenarea modelelor de învățare profundă. Numărul redus de subiecți FTD contribuie direct la dificultatea clasificării acestei clase.
- **Setul holdout mic.** Evaluarea la nivel de subiect s-a realizat pe trei subiecți, ceea ce nu permite formularea unor concluzii statistice generale. Rezultatele holdout sunt ilustrative și complementare metricilor de validare.
- **Trăsături nodale limitate.** Vectorul de trăsături al fiecărui nod conține doar puterea spectrală pe patru benzi. Informații suplimentare, precum coerența, entropia sau trăsăturile de fază, ar putea îmbunătăți capacitatea discriminativă a modelului.
- **Conectivitate PLV pe o singură bandă.** Calculul PLV s-a realizat exclusiv în banda Alpha. Extinderea la multiple benzi ar produce reprezentări de graf mai bogate, la costul unei complexități computaționale mai ridicate.
- **Absența ICA în pipeline-ul implicit.** Curățarea artefactelor prin ICA este dezactivată implicit din cauza riscului de eliminare accidentală a componentelor cerebrale relevante în absența validării manuale. Un protocol automat de detecție a artefactelor ar îmbunătăți calitatea datelor de intrare.

- **Lipsa unui set de test independent.** Evaluarea a utilizat același set de validare atât pentru selecția hiperparametrilor, cât și pentru raportarea metricilor finale, ceea ce poate introduce o supraestimare moderată a performanței raportate.

6.3 Direcții viitoare de dezvoltare

Pe baza concluziilor și limitărilor identificate, sunt propuse următoarele direcții de extindere a lucrării:

1. **Extinderea setului de date și augmentarea.** Includerea unor seturi de date publice suplimentare (de exemplu, TUEG, CAUEEG) și aplicarea unor tehnici de augmentare mai avansate (mixup la nivel de graf, perturbări ale structurii grafului) ar putea crește robustețea și capacitatea de generalizare a modelelor.
2. **Conectivitate multi-bandă.** Construirea unor grafuri multi-layer în care fiecare strat corespunde unei benzi de frecvență ar permite capturarea simultană a dinamicii spectrale la diferite scale temporale, o abordare recent investigată în literatura de specialitate [2].
3. **Trăsături nodale îmbogățite.** Adăugarea unor trăsături suplimentare la nivelul nodurilor — precum entropia spectrală, indicii de complexitate (Higuchi, Hjorth) sau trăsături de conectivitate locală — ar putea crește puterea discriminativă a reprezentărilor de graf.
4. **Modele spațio-temporale.** Integrarea unei componente temporale explicite, prin arhitecturi de tip ST-GNN (Spatio-Temporal GNN) sau prin combinarea GNN cu rețele recurente (LSTM, GRU), ar permite exploatarea dinamicii temporale a semnalelor EEG, dincolo de caracterizarea spectrală per fereastră [4].
5. **Tehnici de echilibrare a claselor.** Pentru a aborda dezechilibrul clasei FTD, pot fi utilizate tehnici de supraesantionare la nivel de graf (SMOTE-GNN), generare de date sintetice sau ajustarea dinamică a ponderilor de clasă în timpul antrenării.
6. **Interpretabilitate.** Integrarea unor metode de explicabilitate specifice grafurilor, precum GNNExplainer sau Grad-CAM adaptat pentru GNN, ar permite identificarea electrozilor și conexiunilor cu cea mai mare contribuție la decizia modelului, crescând relevanța clinică a sistemului [1].
7. **Validare clinică.** O evaluare pe un set de date mai mare, cu protocol de validare încrucișată la nivel de subiect și implicarea unor experți clinici în analiza rezultatelor, reprezintă pasul necesar pentru a transforma prototipul actual într-un instrument cu potențial de utilizare în practica medicală.

Bibliografie

- [1] X. Hu, Z. Sun, Y. Nian, Y. Wang, Y. Dang, F. Li, J. Feng, E. Yu, and C. Tao, “Self-explainable graph neural network for alzheimer disease and related dementias risk prediction: Algorithm development and validation study,” *JMIR Aging*, vol. 7, p. e54748, 2024.
- [2] D. Klepl, M. Wu, and F. He, “Graph neural network-based eeg classification: A survey,” *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 32, pp. 1–1, 2023. Disponibil și ca arXiv:2310.02152.
- [3] M. Awais, S. B. Belhaouari, and K. Kassoul, “Graphical insight: Revolutionizing seizure detection with eeg representation,” *Biomedicines*, vol. 12, no. 6, p. 1283, 2024.
- [4] M. A. Rahman, M. I. Khalil, H. Zhou, Y. Guo, Z. Ding, X. Gao, and D. Zhang, “Advancement in graph neural networks for eeg signal analysis and application: A review,” *IEEE Access*, 2025.
- [5] M. Fey and J. E. Lenssen, “Fast graph representation learning with pytorch geometric,” in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [6] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017.
- [7] A. Gramfort, M. Luessi, E. Larson, D. A. Engemann, D. Strohmeier, C. Brodbeck, R. Goj, M. Jas, T. Brooks, L. Parkkonen, and M. S. Hämäläinen, “MEG and EEG data analysis with MNE-Python,” *Frontiers in Neuroscience*, vol. 7, p. 267, 2013.